

程序设计之书

关于命名、形式、系统与方法的个人正典

作者: 望辰
组织: 拉尼亚凯亚
类型: 设计论纲
邮箱: Laniakeaeaea@Gmail.Com
版本: 1.0.0
日期: 14/05/2026

目录

文档目录

表格目录	II
插图目录	III
代码目录	IV
公式目录	VI
1 命名约定	1
1.1 通用代码命名约定	1
1.2 理论基础	1
1.3 分隔符规范	1
1.4 与大小写体系的比较	2
1.5 片段质量标准：S-I-D 原则	2
1.6 A/HC/LC 模式映射	2
1.7 上下文去重	3
1.8 动作语义	3
1.8.1 布尔前缀	3
1.8.2 范围前缀	3
1.9 后缀规范	4
1.10 语言级应用	4
1.11 文件类型标注	4
1.12 视觉类型系统	4
1.13 命名空间约定	5
1.14 层级经济与常见错误	5
2 通用语言与领域建模	6
2.1 命名不一致的成本	6
2.2 起点：DDD 中的通用语言	6
2.3 逻辑与行为实体：业务核心词汇	6
2.4 视图与交互实体：桌面客户端术语	6
2.5 通信与连接实体：跨进程词汇	7

2.6	动词与动作规格：精确语义分级	8
2.7	版本与生命周期实体：自说明阶段词汇	8
2.8	词典维护原则	8
2.9	回连第 1 章：从“叫什么”到“怎么写”	9
3	代码规范与质量工程	10
3.1	基础与来源	10
3.2	可视化注释规范	10
3.3	JSDoc 契约系统	10
3.4	命名符合性	13
3.5	逻辑与架构	13
3.6	环境与安全	13
3.6.1	Electron 安全铁三角	14
3.6.2	单一事实源	14
3.7	语言细节	14
4	色彩与主题设计	16
4.1	问题：为什么颜色设计困难	16
4.2	系统底色	16
4.3	主题自适应色彩系统	17
4.4	对比理论：WCAG 与 APCA	17
4.5	不对称高亮问题	17
4.6	感知明度与色彩空间	18
4.7	色彩 Token 系统与命名	19
4.8	调色板生成方法	19
4.9	回连前三章	20
5	设计模式	21
5.1	概览与分类	21
5.1.1	为什么需要设计模式	21
5.2	创建型模式：控制对象生成	21
5.2.1	Singleton	21
5.2.2	Registry	22
5.2.3	Factory Method	22
5.2.4	IoC Container	22
5.3	结构型模式：控制对象组织	22

5.3.1 Composite	22
5.3.2 Bridge	23
5.3.3 Adapter	23
5.3.4 Proxy	23
5.3.5 Facade	23
5.4 行为型模式：控制对象协作	23
5.4.1 Command	23
5.4.2 Observer 与 Pub-Sub	23
5.4.3 State	24
5.4.4 Memento	24
5.4.5 Template Method	25
5.4.6 Mediator	25
5.5 并发与资源控制模式	25
5.6 架构模式：模块与边界组织	25
5.7 综合映射	25
5.8 模式选择原则	26
5.9 回连前两章	26
6 程序与架构	28
6.1 设计目标与总原则	28
6.1.1 架构首先是一张所有权地图	28
6.1.2 物理结构必须服从运行时职责	28
6.1.3 当前术语优先于历史原型标签	28
6.1.4 一个运行时核心，多个内部模块	29
6.1.5 本章读的是职责，不是文件夹	29
6.2 宏观架构	29
6.2.1 顶层分区	29
6.2.2 职责流如何贯穿系统	29
6.2.3 工程支撑层	29
6.3 主要结构域	30
6.3.1 Elements	30
6.3.2 Component 与 Element 的边界	30
6.3.3 Functions	31
6.3.4 Layers	31
6.4 Functions 层详解	31
6.4.1 系统能力簇	31

6.4.2	业务 functions 是领域边界	32
6.4.3	Function 内部的角色分工	32
6.4.4	运行时启动必须有明确拥有者	32
6.5	Layers 与视图系统	32
6.5.1	Layers 是壳层级容器	32
6.5.2	Layout 是结构问题，不是业务问题	32
6.5.3	工作台布局是一套结构语法	32
6.5.4	布局选择、删减与显隐策略	33
6.5.5	覆盖层表面仍属于同一布局家族	34
6.5.6	视图路由把壳层连接到内容	35
6.6	依赖与放置规则	35
6.6.1	允许的依赖压力	35
6.6.2	放置规则	35
6.6.3	常见架构违例	36
6.7	与相邻章节的关系	36
7	交互系统与命令架构	37
7.1	设计目标与总原则	37
7.1.1	无逻辑 UI	37
7.1.2	输入多态	37
7.1.3	原子能力	37
7.1.4	显式解耦	37
7.2	逻辑架构	38
7.2.1	核心三元组	38
7.2.2	交互流水线	38
7.2.3	作用域解析	38
7.3	命令协议	39
7.3.1	基础接口	39
7.3.2	Command ID 约定	39
7.3.3	执行数据流	40
7.3.4	Command Context 是执行封套	40
7.3.5	上下文边界规则	40
7.3.6	示例：上下文组装	41
7.4	输入映射与上下文路由	41
7.4.1	一个意图，多种触发	41
7.4.2	作用域感知绑定	41

7.4.3	区分 Scope 与 Context	41
7.5	智能发射器与快捷键服务	42
7.5.1	智能发射器	42
7.5.2	快捷键服务	42
7.6	实现指南	43
7.6.1	示例：添加 <code>file:print</code>	43
7.7	工程约束与禁令	43
7.7.1	必须遵守	43
7.7.2	严格禁止	44
7.8	与相邻章节的关系	44
7.9	可借鉴的参考思想	44
8	状态管理与核心服务	46
8.1	设计目标与总原则	46
8.1.1	逻辑必须脱离具体 UI	46
8.1.2	切换必须视为事务	46
8.1.3	状态不是倾倒场	46
8.1.4	所有权必须显式	46
8.2	服务内核	47
8.2.1	服务是运行时单元	47
8.2.2	为什么服务必须由内核拥有	47
8.3	文件系统服务	47
8.3.1	虚拟文件系统抽象	47
8.3.2	文件生命周期应视作状态机	48
8.4	上下文状态管理器	48
8.4.1	它解决的问题	48
8.4.2	视图状态快照模式	48
8.4.3	切换是三阶段事务	48
8.4.4	可恢复状态不等于持久业务状态	49
8.5	配置服务	49
8.5.1	配置是一组经过校验的数据	49
8.5.2	分层配置优先级	49
8.5.3	热更新与安全边界	50
8.6	反馈服务	50
8.6.1	统一系统反馈	50
8.6.2	消息严重级策略	50

8.6.3	Store 与 Renderer 的分离	50
8.7	实现指南	50
8.7.1	让编辑器具备可恢复能力	50
8.8	工程约束与禁令	50
8.8.1	必须遵守	51
8.8.2	严格禁止	51
8.9	与相邻章节的关系	52
8.10	可借鉴的参考思想	52
9	并发控制与资源管理	53
9.1	设计目标与总原则	53
9.1.1	特权能力必须有单一拥有者	53
9.1.2	进程边界就是架构边界	53
9.1.3	并发控制是选择性的，不是全局性的	53
9.1.4	重复触发必须无害	53
9.2	运行时边界：Main、Renderer 与 Utility	53
9.2.1	为什么桌面运行时是多进程	54
9.2.2	主进程职责	54
9.2.3	渲染进程职责	54
9.2.4	辅助进程是安全阀	54
9.3	窗口管理模型	54
9.3.1	窗口注册表是真相来源	54
9.3.2	已有窗口优先聚焦	55
9.3.3	父子关系与模态关系	55
9.3.4	生命周期清理	55
9.4	独占资源控制	55
9.4.1	为什么原生对话框需要串行化	55
9.4.2	对话框守卫契约	55
9.4.3	锁键设计	55
9.4.4	忽略、合并或排队	56
9.5	IPC 路由与校验	56
9.5.1	共享通道契约	57
9.5.2	主进程集中路由	57
9.5.3	sender 校验是强制项	57
9.5.4	广播与请求-响应必须区分	57
9.6	生命周期协调与故障围堵	58

9.6.1	单实例协调	58
9.6.2	优雅关闭是一笔事务	58
9.6.3	崩溃可见性与恢复边界	58
9.7	实现指南	58
9.7.1	Create-or-Focus 窗口策略	58
9.7.2	原生对话框入口守卫	58
9.8	工程约束与禁令	58
9.8.1	必须遵守	59
9.8.2	严格禁止	59
9.9	与相邻章节的关系	59
9.10	可借鉴的参考思想	60
10	算法与复杂度控制	61
10.1	设计目标与总原则	61
10.1.1	算法是运行时契约	61
10.1.2	先保证正确性，再谈微优化	61
10.1.3	复杂度必须服从交互预算	61
10.1.4	局部变化优先增量更新	61
10.2	核心算法问题族	62
10.3	复杂度预算与运行路径	62
10.4	树遍历与层级寻址	62
10.4.1	为何树结构会反复出现	62
10.4.2	遍历顺序必须服从语义	63
10.4.3	只有遍历还不够	63
10.5	搜索、过滤与稳定排序	63
10.5.1	推荐的计算流水线	63
10.5.2	排序必须稳定	63
10.5.3	重复搜索应依赖索引	64
10.6	优先级调度、队列策略与有界历史	64
10.6.1	为什么 FIFO 常常不够	64
10.6.2	抢占式优先级队列	64
10.6.3	丢弃、合并、排队、串行是不同策略	64
10.6.4	有界历史宜使用环形缓冲区	64
10.7	DAG 拓扑与依赖排序	64
10.7.1	为什么需要拓扑顺序	65
10.7.2	循环检测是必需项	65

10.7.3 启动顺序应被声明为数据	65
10.8 增量计算与局部重建	65
10.8.1 脏标记与版本戳	65
10.8.2 比例状态往往比绝对值更稳定	65
10.8.3 局部重建优于全局失效	65
10.9 算法策略需要升级的信号	65
11 版本控制与发布工程	67
11.1 版本号混乱的代价	67
11.2 基础：Semantic Versioning 2.0.0	67
11.2.1 核心规则	67
11.2.2 预发布标识	68
11.2.3 构建元数据	68
11.3 生命周期阶段：从 SemVer 到通用语言	68
11.3.1 五阶段版本格式	69
11.3.2 阶段词汇的自说明性	69
11.3.3 阶段推进规则	69
11.4 构建产物命名	69
11.5 更新日志约定	70
11.5.1 分类词汇	70
11.5.2 格式规范	70
11.6 发布流水线	70
11.6.1 版本号决策流程	70
11.6.2 分支到环境的映射	71
11.7 制品分发决策树	71
11.8 版本号工具化实现	71
11.9 维护原则	72
11.9.1 多版本分支下的热修复策略	72
11.9.2 版本号绝不复用	73
11.9.3 破窗警告	73
11.10 回连第 2 章：版本字符串的两层结构	74
A 参考文献	I

索引

表格目录

1.1	标识符大小写约定	1
1.2	分隔符层级与语义	2
1.3	A/HC/LC 映射示意	3
1.4	常见动作词的语义边界	3
1.5	布尔前缀约定	4
1.6	范围类前缀约定	4
1.7	路径镜像原则	5
2.1	逻辑实体词汇	6
2.2	视图实体词汇	7
2.3	Element 的判定标准	7
2.4	Container 类型示意	7
2.5	通信实体词汇	8
2.6	通信动作动词	8
2.7	资源控制动作动词	8
2.8	生命周期阶段词汇	9
2.9	第 1 章与第 2 章的关系	9
3.1	规范来源与影响	10
3.2	常用 JSDoc 扩展标签	11
3.3	命名符合性检查	13
4.1	色彩系统的基础来源	16
4.2	基础色与语义锚点	16
4.3	为什么简单反相会失败	17
4.4	跨主题的语义等价	17

4.5	共享语义层级	17
4.6	WCAG 2.1 对比度门槛	18
4.7	按元素类别制定对比策略	18
4.8	不对称高亮的典型现象	18
4.9	高亮策略对比	18
4.10	HSL 的错位问题	18
4.11	OKLCH 等明度控制的优势	19
4.12	色彩 token 层级	19
4.13	token 命名建议	19
4.14	调色板工具链	19
4.15	手工调色流程	19
4.16	色彩章节与前置章节的桥接	20
5.1	模式分类与覆盖范围	21
5.2	项目中的单例实例	21
5.3	项目中的注册表实例	22
5.4	项目中的 Composite	22
5.5	项目中的命令实例	24
5.6	Observer 与 Pub-Sub 的使用场景	24
5.7	Memento 的应用	24
5.8	Template Method 适用场景	25
5.9	模式与系统职责的综合映射	26
5.10	适合引入模式的信号	26
5.11	三层闭合关系	27
6.1	顶层架构分区	29
6.2	运行时问题与架构所有者映射	30
6.3	支撑子树正规化及其职责	30

6.4	视图层级与结构所有权映射	31
6.5	系统能力簇及其架构含义	31
6.6	function 模块内部的典型角色	32
6.7	桌面工作台布局的规范结构区域	33
6.8	布局区域策略：核心必需与可选/可切换表面	34
6.9	依赖压力与放置逻辑	35
7.1	交互层的核心三元组	38
7.2	从输入到执行的交互流水线	38
7.3	命令上下文的典型层次	40
7.4	派发时命令上下文示例	41
7.5	输入多态：同一意图跨多种触发进入系统	41
7.6	同一物理输入在不同作用域下的绑定差异	42
7.7	成熟交互系统中可借鉴的设计思想	45
8.1	典型的虚拟文件系统操作	48
8.2	文件生命周期状态及其要求	48
8.3	视图状态快照中的典型字段	49
8.4	上下文切换的三阶段事务	49
8.5	反馈严重级策略	50
8.6	状态管理与核心服务中可借鉴的设计思想	52
9.1	桌面运行时中的进程所有权	54
9.2	窗口注册表的职责	55
9.3	重复或重叠请求的推荐策略	56
9.4	优雅关闭的阶段	58
9.5	并发控制与资源管理中可借鉴的设计思想	60
10.1	系统中的核心算法问题族	62
10.2	按运行路径划分的复杂度预算	62

10.3	按问题语义选择遍历顺序	63
10.4	重复请求与竞争请求的策略区分	64
11.1	生命周期阶段到 SemVer 的映射（格式与受众）	69
11.2	生命周期阶段到 SemVer 的映射（约束与示例）	69
11.3	更新日志分类词汇	70
11.4	分支、阶段语义与环境映射	71
11.5	语义层与工程层的对应关系	74

索引

插图目录

2.1	视图层级示意	7
5.1	IPC / Pub-Sub 关系示意	24
5.2	Mediator 协调示意	25
5.3	星型内核示意	26
6.1	完整工作台布局轮廓：一种较完整的壳层语法	33
6.2	均衡工作台布局：从完整结构语法中选取较小子集	34
6.3	覆盖层布局：在同一工作台壳层之上的弹窗与对话框表面	35
11.1	阶段推进与回退	69
11.2	版本号决策树	72
11.3	制品分发决策树	73

索引

代码目录

1.1	结构关系与扁平命名	2
1.2	上下文去重示例	3
1.3	DOM / 数据对象的区分示例	4
2.1	视图命名示意	8
3.1	盒状分节头	10
3.2	逻辑分组注释	11
3.3	行尾注解	11
3.4	任务与状态锚点	11
3.5	区域折叠	11
3.6	模块元数据声明	12
3.7	@typedef 类型定义	12
3.8	函数契约示例	12
3.9	泛型模板示例	12
3.10	Guard Clause	13
3.11	策略分发示例	13
3.12	错误处理示例	14
3.13	配置对象示例	14
3.14	逻辑注释示例	14
3.15	HTML 注释示例	15
4.1	双通道高亮思路	18
5.1	IoC 与传统依赖管理对比	22
5.2	VFS 适配器结构	23
5.3	DialogGuard 作为代理	23

5.4	文件状态机示意	24
5.5	三层闭合示例	27
7.1	基础命令接口	39
7.2	发射器侧的命令声明	42
7.3	快捷键到命令的翻译	42
7.4	打印命令实现	43
7.5	注册并暴露打印命令	43
8.1	基础服务契约	47
8.2	编辑器快照导出与恢复契约	51
9.1	原生对话框的独占守卫	56
9.2	单例辅助窗口的主进程 IPC 路由	57
9.3	单例窗口的 create-or-focus 策略	59
11.1	SemVer 核心格式	68
11.2	SemVer 预发布标识语法	68
11.3	构建元数据语法	68
11.4	构建产物命名约定	70
11.5	更新日志格式示例	71
11.6	版本数据源与组装方式	74

索引

公式目录

4.1	WCAG 对比度公式	17
4.2	WCAG 相对亮度定义	17

第 1 章

命名约定

命名不是代码表面的装饰，而是结构与语义的第一层投影。一个稳定系统必须让文件名、目录名、类型名、变量名与动作名共同构成同一套语言，否则维护者只能不断在上下文中猜测真正含义。本章给出本书使用的命名体系：它既规范“怎么写”，也约束“为什么这样写”。

1.1 通用代码命名约定

在代码内部，变量、常量、函数与类仍需要遵守语言级命名习惯；在文件与目录层面，还需要额外的结构分隔规则。这两层并不冲突，而是互补。

表 1.1: 标识符大小写约定

类别	约定	例子
普通变量	camelCase	activeTabId, userConfig
布尔变量	camelCase + is/has/can/should	isEditable, hasPermission
全局常量	UPPER_SNAKE_CASE	MAX_RETRY_COUNT
类名	PascalCase	TabManager, UserRepository
函数名	camelCase 动宾结构	getUserById, calculateTotal

在这一层，大小写约定解决的是语言可读性；但一旦跨入文件系统与模块结构，大小写远远不够。

1.2 理论基础

主流命名法如 camelCase、PascalCase 与 snake_case，都存在同一个结构性缺陷：它们倾向于用同一种分隔机制表达不同层级的关系。这样写出来的长名称虽然可读，却无法机械地恢复原始层次。

本章采用的核心思想是：**不同层级，应使用不同分隔符**。这让名字既能被人读懂，也能被工具稳定解析。

1.3 分隔符规范

本系统定义三种层级固定的分隔符：

一般模式为：

代码 1.1: 结构关系与扁平命名

JSON

```
1 form.variant.card
2 form.domain.user
3
4 card-variant__form
5 user-domain__form
```

表 1.2: 分隔符层级与语义

分隔符	名称	语义层级
-	连字符	单一语义单元内部的词连接
_	单下划线	属性、维度或侧面的分离
--	双下划线	属性集合与主体的分离

```
[attribute-1]_[attribute-2]_...__[subject]
```

这种写法的意义在于：主体只有一个，属性可以有多个，而每一层关系都能被稳定识别。

1.4 与大小写体系的比较

本体系不是否定 camelCase 或 PascalCase，而是指出：大小写适合语言内部标识符；文件系统、目录树与跨模块制品需要额外结构线索。以双下划线标记主体，可以显著降低长文件名的语义歧义。

1.5 片段质量标准：S-I-D 原则

良好的命名片段应满足 S-I-D 三原则：

- Specific** 使用足够具体的词，避免 data、item 这类空洞容器词。
- Identifiable** 名称必须能被直接搜索、讨论与复述，避免晦涩缩写。
- Differentiated** 名称之间必须具有足够差异，避免 userService、userManager、userHandler 在无语义区分时并存。

S-I-D 约束的是命名片段的质量，而不是形式。形式正确但语义空洞的名字，同样是坏名字。

1.6 A/HC/LC 模式映射

结构上可以把名称拆成三层：A 表示属性集合，HC 表示高层语境，LC 表示局部限定。是否写成完整三层，应由歧义程度决定，而不是机械补全。

表 1.3: A/HC/LC 映射示意

层	说明
A	直接描述对象当前职责或属性
HC	提供较高层语境，防止跨域歧义
LC	在局部上下文中做最终限定

1.7 上下文去重

目录上下文、模块上下文与类型上下文本身已经提供语义时，文件名或变量名就不应重复搬运同一信息。

代码 1.2: 上下文去重示例

Examples

```
1 components/button/button__element.ui.js    // good
2 components/button/button-component__element.ui.js // redundant
```

去重不是删信息，而是把信息放回最合适的层级。

1.8 动作语义

动词决定行为边界。get、load、fetch、create、open、toggle 不应被混用，它们各自暗示了不同副作用与调用预期。

表 1.4: 常见动作词的语义边界

动词	推荐语义
get	读取已存在数据，不引入外部副作用
load	从外部源装入资源，可能伴随 I/O
create	生成新实体
open	打开窗口、文件或对话框等可见表面
toggle	在两个稳定状态间切换

布尔前缀

布尔命名应让真假语义一眼可读。

范围前缀

涉及范围或阈值时，应优先使用可直接解释的前缀。

表 1.5: 布尔前缀约定

前缀	典型含义
is	当前状态是否成立
has	是否拥有某属性或资源
can	是否具备执行能力
should	是否建议执行某动作

表 1.6: 范围类前缀约定

前缀	用途
min	最小边界
max	最大边界
default	默认值
current	当前状态

1.9 后缀规范

后缀不应滥用。只有当后缀真正承担类型区分作用时，才值得保留，例如 `Manager`、`Registry`、`Adapter`、`Command` 等。

1.10 语言级应用

TypeScript、Python、SQL、文件路由与后端脚本都可以共享本章的结构原则，但应服从各自生态的最小习惯。例如 `Next.js` 路由目录偏好 `kebab-case`，Python 模块偏好 `snake_case`，而类名仍应使用 `PascalCase`。

1.11 文件类型标注

当一个主体可能跨多个环境存在时，文件类型标注可以承担“环境差异”的角色，例如 `.ui.js`、`.server.ts`、`.spec.ts`。它们是属性，不是主体，所以应附着在主体之后。

1.12 视觉类型系统

视图系统中的命名不仅要表达功能，还要表达层级身份。`Element`、`Component`、`Page`、`Layout` 与 `Container` 的边界一旦模糊，命名就会立刻退化成感觉用语。

代码 1.3: DOM / 数据对象的区分示例

JavaScript

```
1 const searchInputElement = root.querySelector('.search-input');
2 const searchQuery = state.searchQuery;
```

像 `El`、`Node`、`Ref` 这类标记，在局部代码里有助于立即区分“这是 `DOM` 引用”还是“这是纯数据”。

1.13 命名空间约定

名称还应镜像路径与归属关系。目录树既然已经表达出模块位置，命名空间就应与之对应，而不应另起一套平行体系。

表 1.7: 路径镜像原则

原则	说明
模块名镜像路径	路径是物理归属，名字是逻辑投影
局部上下文不重复	已由目录承载的信息不再在文件名中复写
跨域资源显式标注	当模块跨进程或跨环境时，必须在名称中显式可见

1.14 层级经济与常见错误

层级不是越多越好。名称应在消除歧义所需的最小复杂度上停止。最常见错误包括：重复目录上下文、使用语义空洞的泛词、滥用后缀、在同一项目中混用多套动作词，以及用缩写掩盖真正语义。

命名是一种架构工具。一个系统若连最小单位的名字都无法稳定表达，其更高层的组织几乎不可能长期稳定。

第 2 章

通用语言 与 领域建模

命名规则解决“怎么写”，通用语言解决“到底该叫它什么”。一旦一个系统在核心概念上没有统一词汇，团队成员就会在讨论、设计、实现与文档之间不断发生语义漂移。本章的任务，是为整个程序设计体系建立一套统一词典，使每个词都拥有稳定边界。

2.1 命名不一致的成本

如果同一个概念在不同文件里被叫作 `panel`、`view`、`window`、`container`，维护者就无法从名字判断真实边界。命名不一致不仅增加沟通成本，也直接破坏结构判断、搜索定位与接口设计。

2.2 起点：DDD 中的通用语言

领域驱动设计给出的关键启示是：一个概念，在一个边界内应当只有一个名字；一个名字，也应只指向一个概念。这不是写作偏好，而是建模纪律。

2.3 逻辑与行为实体：业务核心词汇

服务、命令、管理器、注册表、适配器、状态、配置，这些都不是可互换词。它们分别指向不同职责：

表 2.1: 逻辑实体词汇

术语	边界
Service	承载长寿命能力与业务逻辑，不直接等同于 UI
Command	表示一次原子用户意图的执行入口
Manager	负责协调某类资源或对象生命周期
Registry	负责按 key 查找已存在实例，不负责创建
Adapter	负责把外部接口转换为系统内部统一接口
State	表示某对象或流程在某一时刻的合法状态

这些词一旦混用，后续章节关于模式、架构与并发的论证都会失去根基。

2.4 视图与交互实体：桌面客户端术语

本书对桌面视图系统采用五层结构：`Container`、`Layout`、`Page`、`Component`、`Element`。它们的边界如下：

表 2.2: 视图实体词汇

术语	定义
Container	壳层级宿主表面，承载整个工作区域或窗口
Layout	空间结构骨架，决定槽位与区域关系
Page	可独立导航的完整界面
Component	由多个 Element 组成的可复用语义组合
Element	最小不可再拆的 UI 粒子

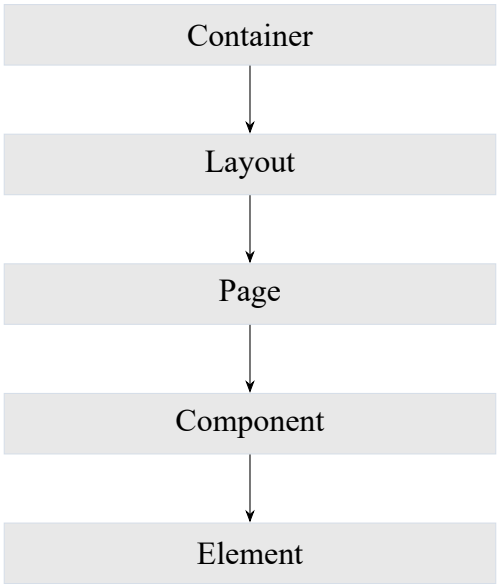


图 2.1: 视图层级示意

表 2.3: Element 的判定标准

维度	判定方式
可拆分性	是否还能进一步拆成多个有独立职责的视觉单元
复用粒度	是否以最小粒度在多个地方复用
语义密度	是否只表达一种局部语义

当一个对象已经由多个 Element 组成并具备明确语义，它就不应再被称为 Element。

表 2.4: Container 类型示意

类型	例子
Window container	主窗口、设置窗口、子窗口
Panel container	左侧导航面板、右侧检查器、底部控制台
Overlay container	对话框、遮罩式浮层

2.5 通信与连接实体：跨进程词汇

桌面系统天然存在 main、renderer 与外部资源之间的边界，因此通信词汇必须清楚区分：channel 用于通道，event 用于通知，request/response 用于请求-响应，bridge 用于跨边界 API 暴露。

代码 2.1: 视图命名示意

Examples

```
1 search-bar__component.ui.js
2 settings-page__page.ui.js
3 main-window__container.ui.js
```

表 2.5: 通信实体词汇

术语	定义
Channel	稳定命名的通信通道
Event	一次已经发生的通知
Request	带返回期待的调用意图
Bridge	连接不同运行环境的 API 边界

2.6 动词与动作规格：精确语义分级

名词定义对象边界，动词定义动作边界。对通信类动作，可使用如下区分：

表 2.6: 通信动作动词

动词	推荐语义
emit	从本地发射一个事件
dispatch	派发给已注册处理器
invoke	发起请求-响应式调用
broadcast	向多个接收者扇出通知

表 2.7: 资源控制动作动词

动词	推荐语义
open	打开资源或可见表面
close	关闭资源或结束占用
focus	把已有资源提升到前景
restore	从已保存状态恢复

2.7 版本与生命周期实体：自说明阶段词汇

版本系统需要的不只是数字，还需要清晰阶段词。本书采用五个阶段词：Experimental、Development、Integration、Preview、Candidate。

2.8 词典维护原则

通用语言不是一次性写完的表格，而是必须持续维护的词典。新增术语应先判断其是否真的表达新概念；若只是已有概念的别名，就应被拒绝。

表 2.8: 生命周期阶段词汇

阶段	含义
Experimental Development	高度实验性，仅供核心开发内部试错
Integration	日常开发构建，面向内部开发者
Preview	功能冻结，进入集成测试阶段
Candidate	面向较广用户的预览版本
	接近正式发布的候选版本

2.9 回连第 1 章：从“叫什么”到“怎么写”

第 1 章与第 2 章共同完成命名系统：第 2 章先定义语义边界，第 1 章再规定书写形式。

表 2.9: 第 1 章与第 2 章的关系

章节	核心问题	回答
第 1 章	这个名字如何写	分隔符、大小写、上下文去重、动作形式 术语边界、视图层级、通信语义、生命周 期词汇
第 2 章	这个对象到底该叫什么	

没有第 2 章，第 1 章只会得到形式正确却语义含混的名字；没有第 1 章，第 2 章则只是一份词典，无法转化为可执行规则。

第 3 章

代码规范 与 质量工程

命名解决结构可读性，通用语言解决术语边界，而代码规范则负责让这些约束真正落到工程实践中。本章面向原生 JavaScript 与 Electron 场景，重点讨论注释可视结构、JSDoc 契约、逻辑组织、安全边界与配置治理。

3.1 基础与来源

本章吸收了 Clean Code、Google JS Style Guide、Airbnb Style Guide 与 Electron 官方安全建议，并把它们收束为适合本体系的工程标准。

表 3.1: 规范来源与影响

来源	影响
Clean Code	小函数、guard clause、异常优先、避免歧义注释
Google JS Style Guide	JSDoc 契约系统与类型注释风格
Airbnb JS Style Guide	TODO/FIXME 维护锚点、布尔命名前缀
Electron Security	nodeIntegration=false、contextIsolation=true、sandbox=true

3.2 可视化注释规范

在缺少强类型与 IDE 结构辅助的环境里，注释不仅解释意图，还承担文件级逻辑地图的作用。理想代码应尽量自说明，但在原生 JS 环境中，标准化视觉标记仍是有价值的结构补丁。

代码 3.1: 盒状分节头

JavaScript

```
1  /* =====  
2   IPC Event Handlers  
3   ===== */
```

行尾注解应解释“为什么”，而不是重复“做了什么”。

3.3 JSDoc 契约系统

在原生 JavaScript 中，JSDoc 同时承担静态检查与文档契约两种角色。本体系要求所有公开模块与重要函数都提供最小契约说明。

代码 3.2: 逻辑分组注释

JavaScript

```
1  /* User Authentication */
2  // ... login logic ...
3
4  /* Data Synchronization */
5  // ... sync logic ...
```

代码 3.3: 行尾注解

JavaScript

```
1  const HIT_AREA_PADDING = 10; /* Increased for touch devices */
```

代码 3.4: 任务与状态锚点

JavaScript

```
1  // TODO: Implement pagination for search results
2  // FIXME: Memory leak when repeatedly toggling sidebar
3  // HACK: Chrome SVG rendering bug workaround
4  // NOTE: Renderer uses command indirection by design
```

代码 3.5: 区域折叠

JavaScript

```
1  // #region Status Code Mappings
2  const STATUS_MAP = {
3    // ... 50+ entries ...
4  };
5  // #endregion
```

表 3.2: 常用 JSDoc 扩展标签

标签	用途
@throws	声明异常
@template	声明泛型类型参数
@example	给出使用示例
@readonly	标记只读属性
@enum	标记枚举类型

代码 3.6: 模块元数据声明

JavaScript

```
1 /**
2  * @module elements/button
3  * @file button__element.ui.js
4  * @description Generic button Element
5  * @environment Renderer (DOM)
6  */
```

代码 3.7: @typedef 类型定义

JavaScript

```
1 /**
2  * @typedef {Object} ButtonConfig
3  * @property {string} label
4  * @property {boolean} [isDisabled=false]
5  * @property {'primary' | 'secondary'} variant
6  */
```

代码 3.8: 函数契约示例

JavaScript

```
1 /**
2  * @param {string} userId
3  * @returns {Promise<UserConfig>}
4  * @throws {Error}
5  */
6 async function fetchUserConfig(userId) { /* ... */ }
```

代码 3.9: 泛型模板示例

JavaScript

```
1 /** @template T */
2 function identity(value) {
3   return value;
4 }
```

3.4 命名符合性

本章中的所有代码示例都应回连到第 1 章与第 2 章：形式上符合结构命名规则，语义上符合统一词汇边界。

表 3.3: 命名符合性检查

检查项	要求
形式正确	分隔符、大小写、后缀与上下文去重正确
语义正确	术语与职责边界一致
环境正确	文件名能体现其运行环境或资源类型

3.5 逻辑与架构

代码结构应直接服务于逻辑清晰度与架构边界。优先使用 `guard clause` 减少嵌套，用策略分发代替长链条件，用异常显式表达真正的错误路径。

代码 3.10: Guard Clause

JavaScript

```
1 function saveFile(file) {
2   if (!file) throw new Error('file is required');
3   if (file.isReadOnly) return false;
4   return fileService.write(file.path, file.content);
5 }
```

代码 3.11: 策略分发示例

JavaScript

```
1 const STRATEGY_MAP = {
2   text: saveTextFile,
3   binary: saveBinaryFile
4 };
```

3.6 环境与安全

Electron 的最小安全三角是:`nodeIntegration=false`、`contextIsolation=true`、`sandbox=true`。凡是跨越这三条边界的便捷写法，都应被视为用安全性换局部省事。

Electron 安全铁三角

Main 拥有特权资源，renderer 只通过受限桥接访问能力。任何把系统 API 直接暴露给 renderer 的做法，都在破坏架构边界与安全边界的同构关系。

代码 3.12: 错误处理示例

JavaScript

```
1  try {  
2    await command.execute(context);  
3  } catch (error) {  
4    feedbackService.error(error.message);  
5  }
```

单一事实源

配置不应散落在多个文件常量中。应当有一个单一配置对象，承担默认值、环境覆盖与运行时读取。

代码 3.13: 配置对象示例

JavaScript

```
1  export const APP_CONFIG = {  
2    theme: 'system',  
3    locale: 'zh-CN',  
4    zoomLevel: 1  
5  };
```

3.7 语言细节

HTML、CSS、JavaScript 与模板语言中的注释风格可以不同，但都应遵守同一条原则：帮助读者快速识别结构与边界，而不是重复显而易见的实现细节。

代码 3.14: 逻辑注释示例

JavaScript

```
1  // NOTE: command layer stays stateless by design
```

代码规范的终点，不是追求“像样板一样整齐”，而是让阅读、扩展、排错与重构都沿着同一套边界发生。

代码 3.15: *HTML* 注释示例**HTML**

```
1 <!-- Sidebar: project navigation and filters -->
```


第 4 章

色彩 与 主题设计

前几章分别定义了名字的写法、术语的边界与代码的组织方式；本章则回答视觉层的问题：在深色与浅色主题之间，如何让同一语义色保持可读性、一致性与可访问性。颜色在软件中不是装饰，而是功能界面的一部分。

4.1 问题：为什么颜色设计困难

颜色设计之所以容易失败，是因为它同时受到感知、上下文与语义三方面影响。相同的物理亮度不一定产生相同的主观亮度；同一颜色在深色背景与浅色背景上会产生不同感受；而在 UI 里，颜色又天然承载功能角色。

表 4.1: 色彩系统的基础来源

领域	关键来源	贡献
可访问性标准	WCAG 2.1, APCA	提供最低对比要求与更接近感知的对比模型
色彩科学	CIE 1931, CIELAB	提供亮度与均匀性的基础理论
现代色彩空间	OKLCH	提供更稳定的色相与明度控制
设计系统	Material 3, Radix, Leonardo	提供 token 化与主题自适应方法

4.2 系统底色

本体系采用七个彩色锚点加两个中性色锚点，足以覆盖大多数 UI 语义，而不会让颜色数量膨胀到难以区分。

表 4.2: 基础色与语义锚点

基础色	语义角色	典型用途
Red	危险 / 错误	删除、失败、不可逆警告
Orange	警示	注意、风险、待确认
Yellow	提示	暂挂状态、说明性提示
Green	成功	完成、通过、安全
Cyan	次级信息	特殊状态、辅助信息
Blue	主动作 / 链接	主要交互、链接、信息
Purple	强调 / 品牌	品牌强调、高级能力
Neutral dark / light	表面与对比基准	背景、文字、边界

4.3 主题自适应色彩系统

本系统的核心规则是：**token 名称跨主题保持稳定，实际数值随主题变化**。这意味着 Blue-100 在深色与浅色主题下可以是不同十六进制值，只要它们承担的是同一语义层级。

表 4.3: 为什么简单反相会失败

场景	失败方式
浅色模式的淡蓝直接复用到深色模式 只反转 HSL 亮度 深浅主题共用同一个 hex	与深色背景融在一起，失去层次 颜色失去原本语义气质 无法同时满足对比与语义稳定

表 4.4: 跨主题的语义等价

上下文	数值	解释
浅色模式	#D0E4FF	适用于近白背景上的浅蓝表面
深色模式	#1A3A6B	适用于深灰背景上的深蓝表面
共享层级	Blue-100	同一语义层级，而非同一数值

表 4.5: 共享语义层级

层级	用途
50 / 100	最浅表面与高亮底色
200 / 300	交互状态与边界
400 / 500	常规前景与说明文本
600 / 700	强调前景与活跃状态
800 / 900	深色表面与靠近背景的重色

4.4 对比理论：WCAG 与 APCA

WCAG 仍然是最低验收线，APCA 则更适合做精细优化。

公式 4.1: WCAG 对比度公式

$$\text{Contrast ratio} = \frac{L_{\text{lighter}} + 0.05}{L_{\text{darker}} + 0.05}$$

公式 4.2: WCAG 相对亮度定义

$$L = 0.2126 \cdot R_{\text{linear}} + 0.7152 \cdot G_{\text{linear}} + 0.0722 \cdot B_{\text{linear}}$$

4.5 不对称高亮问题

深浅主题中，高亮并非镜像关系。深色模式常需要更亮、更饱和的强调色；浅色模式则需要更深、更稳重的强调色，才能达到同样“被看见但不刺眼”的效果。

表 4.6: WCAG 2.1 对比度门槛

等级	普通文本	大文本
AA	4.5:1	3:1
AAA	7:1	4.5:1

表 4.7: 按元素类别制定对比策略

元素类别	最低要求	策略
文本与图标	4.5:1, 长读优先追求 AAA	先确保可读, 再谈风格
表单边框与输入区	3:1 边框, 4.5:1 字体	把边界识别与文字可读分开处理
按钮	文字与按钮面均需可区分	控件与其标签同时可见
容器与卡片	可适当低于文字对比	结合边框、阴影与层级关系共同表达

表 4.8: 不对称高亮的典型现象

现象	说明
深色界面中的高亮偏暗	容易被背景吞没
浅色界面中的高亮过亮	容易显得噪声过强
同一饱和度跨主题复用	语义强度不一致

代码 4.1: 双通道高亮思路

CSS

```
1 --accent-fg: var(--blue-600);
2 --accent-bg: var(--blue-100);
```

表 4.9: 高亮策略对比

策略	特点
单色高亮	简单, 但跨主题稳定性差
前景/背景双通道	更易在不同主题中保持语义等价
语义 token + 状态 token 叠加	最适合复杂系统扩展

4.6 感知明度与色彩空间

HSL 在工程上易用, 但不适合严肃地控制“感知等亮度”。在 UI 主题系统中, 更建议使用 OKLCH 或至少以 LAB 作为验证参考。

表 4.10: HSL 的错位问题

问题	表现
同一 L 值不等于同一感知亮度	黄看起来远亮于蓝
色相变化会改变视觉亮度	难以稳定生成跨色相等层级面板

表 4.11: OKLCH 等明度控制的优势

优势	意义
更稳定的明度控制	有利于构建跨主题的一致语义层级
更自然的色相保持	有利于品牌色与功能色同时存在

4.7 色彩 Token 系统与命名

颜色必须 token 化，避免在组件层直接散落十六进制常量。

表 4.12: 色彩 token 层级

层级	说明
Base tokens	原始色阶，如 blue-100
Semantic tokens	语义角色，如 color-primary-fg
Component tokens	组件特定用色，如 button-primary-bg

表 4.13: token 命名建议

命名形态	用途
blue-500	基础色阶
color-primary-fg	语义前景
surface-panel-bg	宿主表面背景

4.8 调色板生成方法

调色板可以人工生成，也可以借助 Leonardo、contrast.tools 等工具辅助，但最终验收标准仍是语义一致、对比达标、跨主题稳定。

表 4.14: 调色板工具链

工具	用途
Leonardo	根据对比目标生成可调色阶
contrast.tools	快速验证颜色对比
手工校正	处理品牌约束与视觉微调

表 4.15: 手工调色流程

步骤	说明
确定语义角色	先定义 primary / danger / surface 等角色
确定深浅主题基准背景	先固定环境再调色
按对比目标推导层级	再决定 100/500/900 等 token 值
最后做品牌微调	让品牌色服从可读性而非反过来

4.9 回连前三章

色彩系统并不是孤立章节。它回连第 1 章的命名体系、第 2 章的语义边界与第 3 章的工程结构。颜色一旦失去 `token` 命名，就会回到魔法值；一旦失去语义词汇，就会回到“这个蓝看起来不错”的主观讨论。

表 4.16: 色彩章节与前置章节的桥接

前置章节	对本章的约束
第 1 章	规定 <code>token</code> 与文件的书写形式
第 2 章	规定 <code>primary</code> 、 <code>danger</code> 、 <code>surface</code> 等语义边界
第 3 章	规定 <code>token</code> 、配置与主题逻辑如何被组织进代码

第 5 章

设计模式

第 1 章与第 2 章分别定义了名字的形式规则与语义规则，本章则讨论代码组织的规则：项目中的具体模块为何映射到某些经典模式，这些模式如何共同闭合成一个可解释的系统，以及为什么系统最终会以这种方式组织起来。

5.1 概览与分类

为什么需要设计模式

设计模式的价值在于把反复出现的问题及其解法显式化。对大型系统而言，它至少降低三类成本：沟通成本、维护者的心理预期成本，以及面对新问题时“从零发明结构”的成本。但模式并非免费午餐，每个模式都会引入额外层次、更多对象或更远的调用路径。

表 5.1: 模式分类与覆盖范围

类别	核心问题	本章覆盖
创建型	如何控制对象生成	Singleton, Registry, Factory Method, IoC Container
结构型	如何把对象组织成更大骨架	Composite, Bridge, Adapter, Proxy, Facade
行为型	如何协调责任与通信	Command, Observer, Strategy, State, Memento, Template Method, Mediator
并发型	如何安全管理共享资源	Mutex, 优先级队列, 环形缓冲区
架构型	如何组织顶层模块与边界	Modular Monolith, Star Kernel, Parent-Child Hierarchy

5.2 创建型模式：控制对象生成

Singleton

单例模式适合拥有全局唯一性的资源拥有者，例如窗口管理器、应用元数据配置与独占守卫。

表 5.2: 项目中的单例实例

模块	为什么适合作为单例
window__manager.ts	整个应用生命周期中只应存在一个窗口协调入口
app-info__config.ts	应用元数据天然只有一份真实来源
dialog__guard.ts	原生对话框的独占锁管理器应全局唯一

Registry

Registry 负责 key 到实例的查找、登记与反登记。它不创建对象，只回答“已有对象在哪里”。

表 5.3: 项目中的注册表实例

模块	关键行为
window__manager.ts	维护窗口标签到 BrowserWindow 的映射
command__registry.ts	维护 commandID 到命令实例的映射

Factory Method

当具体对象需要在运行时根据上下文延迟决定时，Factory Method 很合适，例如命令的惰性创建或不同编辑器快照对象的派生创建。

IoC Container

IoC Container 的作用，是把依赖装配从对象自身移交给外部内核，从而减少硬编码依赖与循环引用。

代码 5.1: IoC 与传统依赖管理对比

IoC

```
1 class FileService {
2   constructor(configService) {
3     this.configService = configService;
4   }
5 }
```

5.3 结构型模式：控制对象组织

Composite

视图系统天然 是树，因此 Composite 是其基础模式。

表 5.4: 项目中的 Composite

结构	树形形式
视图层级	Container → Layout → Page → Component → Element
窗口层级	主窗口 → 子窗口的父子树

Bridge

在 Electron 架构中，Preload / ContextBridge 同时满足模式需要与安全需要：它把 renderer 的抽象界面与 main 的特权实现解耦。

Adapter

虚拟文件系统通过统一接口屏蔽不同存储后端。

代码 5.2: VFS 适配器结构

VFS

```
1 Local__fs.adapter.ts
2 Memory__fs.adapter.ts
3 Cloud__fs.adapter.ts
```

Proxy

对原生对话框的访问不应直接暴露给任意调用方，而应经由守卫代理。

代码 5.3: DialogGuard 作为代理

DialogGuard

```
1 btn.onClick -> guard.runExclusive('file-open',
2   () => dialog.showOpenDialog(...))
```

Facade

Facade 通过提供高层统一入口，隐藏内部复杂实现，例如反馈系统或命令注册系统都应对外暴露窄接口，而不是把内部实现细节扩散出去。

5.4 行为型模式：控制对象协作

Command

Command 是本体系中最关键的行为模式之一。它把用户请求封装成对象，使排队、日志、宏录制、撤销与统一派发成为可能。

Observer 与 Pub-Sub

两者都解决一对多通知，但耦合方式不同。Observer 更适合对象之间的直接依赖，Pub-Sub 更适合跨边界或跨进程的事件传播。

表 5.5: 项目中的命令实例

模块	职责
base__command.ts	规定统一执行入口与元数据字段
save-file__command.ts	表达保存文件这一原子操作
copy__command.ts	表达复制这一原子操作

表 5.6: Observer 与 Pub-Sub 的使用场景

模式	适用场景
Observer	同一运行时内部的状态变更通知
Pub-Sub	IPC、跨模块广播与解耦事件总线

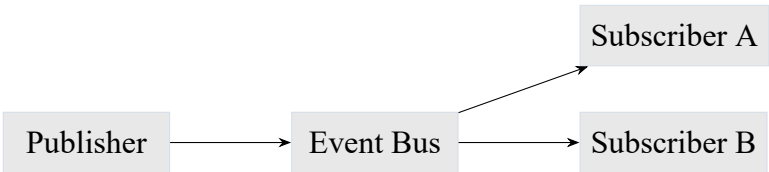


图 5.1: IPC / Pub-Sub 关系示意

State

当对象行为随状态变化而变化时，显式 State 模式优于散乱布尔判断。文件生命周期与窗口状态都属于这类问题。

代码 5.4: 文件状态机示意

State

```
1 virtual -> dirty -> saved
2 saved -> orphaned
3 saved -> readOnly
```

Memento

Memento 负责在不暴露内部实现的前提下保存与恢复状态，这正是编辑器快照与上下文恢复的模式基础。

表 5.7: Memento 的应用

对象	保存内容
文本编辑器	滚动位置、选择区
画布编辑器	相机位置、缩放、激活工具

Template Method

当一类流程的骨架稳定、但局部步骤有差异时，Template Method 很适合，例如统一的导入/导出流水线、保存流程或视图恢复流程。

表 5.8: Template Method 适用场景

流程	可变步骤
文件保存	校验、序列化、写入、反馈
导出流程	格式适配、资源打包、后处理

Mediator

当多个子系统需要协作，但不应形成全互连时，Mediator 负责集中协调。例如工作台布局协调器、命令路由器或反馈总线都可扮演中介者角色。

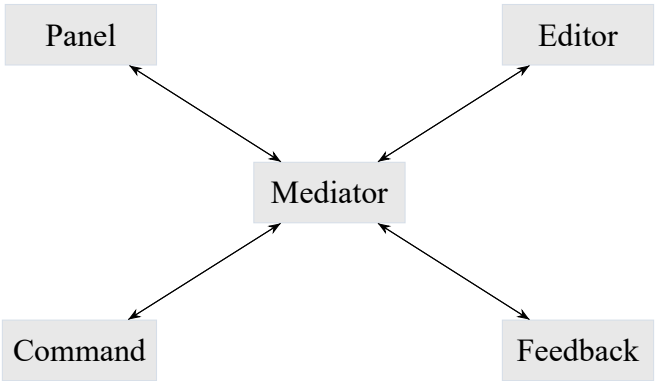


图 5.2: Mediator 协调示意

5.5 并发与资源控制模式

独占资源需要 Mutex，用户反馈需要优先级队列，近期历史更适合环形缓冲区。这些模式虽然不属于经典 GoF 23 模式，但在桌面工程中同样是高频基础设施。

5.6 架构模式：模块与边界组织

本系统在架构层主要体现为模块化单体与星型内核：外部只有一个应用运行时，内部则由多个围绕内核组织的能力模块构成。

5.7 综合映射

模式并不是孤立使用的。单例常与注册表共同管理资源，命令常与观察者和中介者共同构成交互网络，Memento 与 State 共同支撑恢复与生命周期，Bridge 与 Adapter 一起处理跨边界能力与外部接口。

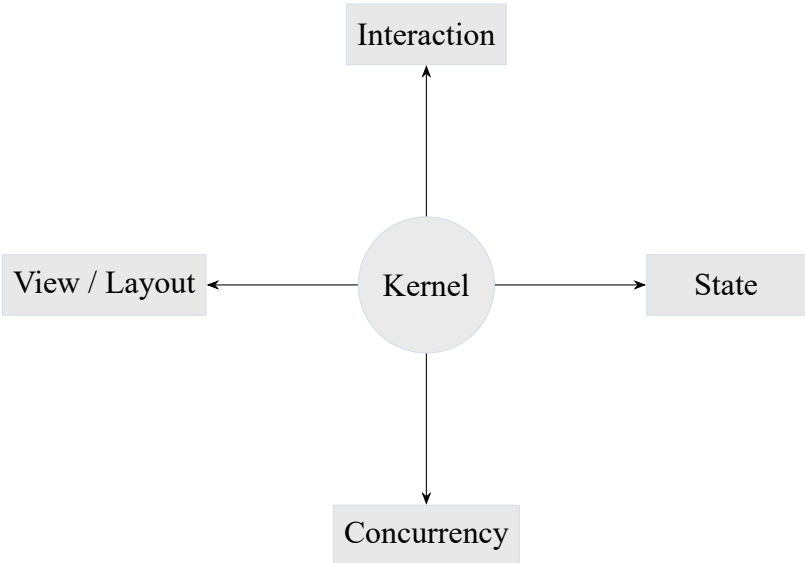


图 5.3: 星型内核示意

表 5.9: 模式与系统职责的综合映射

系统职责	主要模式
命令执行与派发	Command + Registry + Mediator
状态恢复与生命周期	State + Memento + Template Method
窗口与资源控制	Singleton + Registry + Mutex + Proxy
跨进程桥接与外部资源接入	Bridge + Adapter + Facade

5.8 模式选择原则

模式应在结构压力真实存在时引入。以下信号通常说明某个模式值得采用：已有重复问题、职责混乱、对象关系无法被清楚命名、未来扩展方向明确、测试替换困难。反之，如果模式只是为了“看起来正规”，它通常会变成负担。

表 5.10: 适合引入模式的信号

信号	建议
对象创建分散、难以替换	考虑 IoC 或 Factory
资源需要全局唯一	考虑 Singleton 或 Registry
输入与执行强耦合	考虑 Command
状态转移失控	考虑 State
恢复逻辑难以复用	考虑 Memento 或 Template Method

5.9 回连前两章

模式的名字本身也受前两章约束：只有在术语边界稳定、命名形式一致时，模式才真正具备沟通价值。一个被错误命名的 Manager、Service 或 Handler，会立即破坏模式所带来的认知收益。

本章的结论是：模式不是贴在代码表面的标签，而是结构压力被长期识别后的稳定回答。只

表 5.11: 三层闭合关系

层	作用
命名规则	规定名字怎么写
通用语言	规定概念叫什么
设计模式	规定对象如何组织协作

代码 5.5: 三层闭合示例

Example

```
1 save-file__command.ts
2 window__manager.ts
3 dialog__guard.ts
```

要系统继续增长，这些回答就会不断回到架构中心。

第 6 章

程序 与 架构

本章是 [第 2 章](#)、[第 5 章](#)、[第 7 章](#)、[第 8 章](#) 与 [第 9 章](#) 的工程落地篇。第 2 章定义了 Element、Component、Page、Layout、Container 等术语边界；第 5 章给出了模块化单体、星型内核、组合与桥接等模式基础；第 7 到 9 章则界定了命令执行、状态持久化与特权资源协调的所有权。本章的任务，是把这些前置约束落实为一个具有显式边界的项目结构，而不是仅仅罗列目录名。

6.1 设计目标与总原则

程序架构的目的，不是造出一棵“看起来漂亮”的目录树，而是让运行时所有权清晰、依赖方向可理解，并阻止代码库滑向循环引用、壳层文件承载业务逻辑、或初始化路径四处散落的状态。

架构首先是一张所有权地图

一个合格的项目结构，应能快速回答以下工程问题：

- 运行时从哪里开始启动
- 系统级能力由谁持有
- 最小可复用 UI 粒子放在哪里
- 窗口壳层由谁定义
- 新的业务行为可以在何处接入而不污染基础设施

如果结构回答不了这些问题，它就不是架构，只是装饰。

物理结构必须服从运行时职责

代码库的物理布局，应当投影出系统真实的运行时角色。在本项目中，核心关注点是可复用 UI 粒子、可执行能力模块，以及壳层级宿主表面。因此，结构必须先把这些关注点拆开，才允许局部实现细节逐渐积累。

当前术语优先于历史原型标签

历史路径里也许还残留旧标签，但正式文档应采用 [第 2 章](#) 已经建立的当前词汇。因此，本章以 Element、Component、Page、Layout、Container 为视图系统的权威语言，而不是沿用早期

原型中的临时命名。

一个运行时核心，多个内部模块

系统在架构上属于模块化单体：一个可部署应用、一个运行时核心、多个内部模块以及显式边界。目标不是把系统碎裂成大量服务，而是在一个运行时内部维持纪律化分区。

本章读的是职责，不是文件夹

本章不会从“当前碰巧有哪些目录”里倒推出理论，而是用前几章的约束去解释谁拥有启动、命令执行、状态连续性、特权资源控制，以及视图层级的分布。即使把目录树从页面删掉，这些问题仍必须得到回答。

6.2 宏观架构

顶层分区

在最高层，架构应当分为长期资源、工程支撑材料与运行时代码三大类。

表 6.1: 顶层架构分区

区域	主要职责	存在原因
assets/	字体、图标、图片、本地化与其他长期资源	将静态制品与可执行运行逻辑分离
support/	开发参考、计划、日志、验证脚本与维护工具	让工程支撑材料脱离运行时代码路径
src/elements/ src/functions/system/	最小可复用 UI 粒子 跨切面的系统能力	为原子级视图单元保留稳定归属地 集中管理启动、窗口策略、布局编排、交互支持与体验策略
src/functions/ business/	产品特定工作流与领域行为	防止业务规则泄漏进基础设施模块
src/layers/	窗口壳层与结构性宿主表面	让容器层级的所有权显式化

这个分区应当被理解为职责地图，而不是目录导览。目录树之所以重要，只是因为它证明系统已经吸收了这些边界。

职责流如何贯穿系统

当我们用“运行时问题由谁回答”来读结构时，架构会比单看目录名清楚得多。

工程支撑层

support/ 子树不是第四个运行时极点，它是工程支撑层，用来安置参考资料、开发过程文档、验证脚本与维护性工具，使这些内容不进入可执行路径。

表 6.2: 运行时问题与架构所有者映射

运行时问题	架构所有者	与前序章节的关系
应用如何启动、进程策略如何生效	functions/system + layers	延续 第 5 章 的模块化单体与星型内核逻辑，以及 第 9 章 的特权资源边界
用户意图如何从 UI 进入系统	layers → routing → functions	延续 第 7 章 的命令发射与派发模型
运行时状态如何保存、恢复与跨界面传递	functions/system / functions/business	延续 第 8 章 的状态所有权与持久化规则
哪些 UI 单元可复用，哪些是宿主结构	elements / components / pages / containers	延续 第 2 章 的五层视图层级
哪些能力必须保持特权，不得下沉到普通视图	functions/system	延续 第 9 章 的窗口、对话框与资源控制边界

根据 第 1 章 的命名规则，目录名应采用 kebab-case，尽量使用完整单词，并让目录上下文承担语义。因此，支撑子树中的历史缩写应被正规化如下。

表 6.3: 支撑子树正规化及其职责

原型名称	正式名称	职责
dev/ refe/ test/	development/ reference/ test/ development/	开发过程材料的父目录 用于加速开发的代码参考与实现先例 聚焦验证、探针式检查与切片测试脚本
dev/diff/	gap-analysis/	当前实现与目标实现之间差距的分析文档
dev/log/	development/log/	开发日志文档
dev/plan/	development/plan/	开发计划文档
dev/script/	development/script/	非运行时维护脚本，例如资源重命名或清理脚本
dev/spec/	development/standard/	开发标准、约定与规范性约束

其中 test/、log/、plan/、script/ 已是可接受目录类别；真正需要修正的是 dev、refe、diff、spec 这类缩写或语义不完整名称。

6.3 主要结构域

Elements

elements 层承载系统中最小的可复用 UI 粒子。这些单元负责局部视觉表达与局部交互信号发射，但不拥有业务意义，也不持有特权运行策略。这是 第 2 章 中 Element 定义在结构层面的直接后果。

Component 与 Element 的边界

Component 与 Element 的正式定义、判准与示例，应由 第 2 章 负责。本章只保留其结构后果：Component 是视图层级中的可复用语义组合体，而不是项目根树上的一级所有权域；Element 则是最小可复用 UI 粒子。因此，elements/ 可以作为物理家族存在，而 Component 更常以内聚组合的方式存在于 Page、路由视图或 Layer 宿主表面之内。

Functions

functions 层承载具名能力模块。任何行为只要已经大于单一 Element、又比一次临时事件回调更稳定，都应被上提到这里，而不应残留在视图或壳层文件中。

Layers

layers 层承载壳层级宿主表面。它们不是普通 Component，而是其余 UI 得以存在的结构容器。因此，诸如 main-window/、setting-window/、sub-window/ 这样的名称，不只是文件目录，而是在声明这些根节点属于 第 2 章 的 Container / Shell 层级。

表 6.4: 视图层级与结构所有权映射

视图层级	典型物理归属	在本章中的架构含义
Container / Shell Layout	src/layers/ layer 壳层结构 + layout-sys	窗口级宿主表面与壳层所有权 静态空间骨架、槽位几何与宿主级组合
Page Component	路由到达的业务界面 页面内或路由内的语义组合	被壳层承载的完整可导航界面 位于视图层级内部的复用单元，而不是顶层归属根
Element	src/elements/	最小可复用 UI 粒子

6.4 Functions 层详解

系统能力簇

理解 functions/system/ 最有效的方式，不是把它当目录列表，而是把它看成若干必须集中管理的能力簇。

表 6.5: 系统能力簇及其架构含义

能力簇	代表模块	架构含义
启动与窗口策略	app-core-sys, window-sys, dialog-sys	拥有应用启动、窗口生命周期与特权交互边界；直接延续 第 9 章 的资源控制要求
视图宿主与结构编排	view-sys, layout-sys, tab-bar-sys, tooltip-sys	把 layer 壳层变成可导航、可布局、可承载内容的运行时表面；连接 第 2 章 的视图层级与 第 7 章 的交互流水线
环境与体验策略	theme-sys, language-sys, font-sys, zoom-sys, preference-sys	持有必须跨页面、跨窗口、跨会话保持一致的体验规则；直接连接 第 8 章 的持久化边界

这些能力必须集中，否则它们会溶解进 Elements、Pages 或壳层文件中，制造跨界面耦合。

业务 functions 是领域边界

functions/business/ 是产品特定工作流的保留地。即便它在项目早期还很稀疏，它在架构上仍非常关键，因为它阻止一种常见退化：由于基础设施模块已经存在，业务规则就被顺手塞进去。

结合第 7 到 9 章的约束，业务 functions 应当成为以下行为的归宿：它们由命令触发，依赖状态读写，但又不属于特权系统控制。

Function 内部的角色分工

成熟的 function 模块通常需要把编排、执行、配置与外部暴露分离开来。

表 6.6: function 模块内部的典型角色

子结构	角色	意义
index.js	公开入口	定义对外暴露的导入边界
central/	运行时拥有者与协调者	承载生命周期协调或跨子模块所有权
engines/	稳定执行逻辑	实现解析、尺寸计算、渲染支持等具体运行机制
synaptic/	连接器与协调节点	连接 API、管理器与交互表面
specs/	声明式配置	将长期常量与正式配置从活跃执行代码中抽离
views/	系统拥有的视觉资源	保存属于该系统能力本身的 CSS 或视图资源

运行时启动必须有明确拥有者

主进程入口 app-core-sys/central/ 应作为启动权威拥有者。应用就绪、窗口生命周期启动与进程级策略决策应在这里汇聚，而不是散落到无关文件中。初始化分布在任意位置并不叫灵活，只叫含混。

6.5 Layers 与视图系统

Layers 是壳层级容器

layers/ 子树是容器层的物理表达。主窗口、设置窗口、子窗口这类壳层，不是高层业务组件，而是定义“内容可以在何处生长”的宿主结构。

Layout 是结构问题，不是业务问题

布局子系统与壳层 HTML 一起决定标题栏、侧边栏、标签区域、内容区等结构槽位。这些属于 Layout，而非业务 Page。守住这条边界，才能避免业务 workflow 反过来接管窗口几何结构。

工作台布局是一套结构语法

本章配套的三张轮廓图强调一个关键点：桌面工作台的本质结构，并不从品牌样式或某个功能组件开始，而是从一组有限的结构区域开始。这些区域可以被组合、删减、折叠或切换，用

来承载不同产品需要。

表 6.7: 桌面工作台布局的规范结构区域

区域	结构层级	主要职责
Title Bar	Shell 边界	窗口身份、拖拽区域与高层窗口控制
Menu Bar	Shell 命令表面	全局命令族与应用级动作入口
Top Tool Bar	workflow 表面	当前工作上下文中的高频主动作
Top Tool Tray	辅助动作带	次级切换、快捷状态与紧凑状态提示
Left Tool Bar	侧向工具表面	左侧模式切换或工具族选择
Right Tool Bar	侧向工具表面	右侧上下文工具或检查器
Left Panel / Left Sub Panel	Container 宿主区域	导航、文件树、大纲、过滤器与辅助浏览
Right Panel / Right Sub Panel	Container 宿主区域	属性、检查器、预览或上下文细节
Bottom Panel	Container 宿主区域	控制台、日志、时间线、终端或次级信息
Main Area	主内容区域	承载当前活动 Page 及其核心工作内容

这些区域应被理解为结构语法，而不是强制清单。它们的价值在于为壳层组合提供稳定词汇。



图 6.1: 完整工作台布局轮廓：一种较完整的壳层语法

这张完整轮廓图的意义，不是要求每个产品同时实例化全部区域，而是强调这些区域具备稳定结构身份，因此应由布局层显式拥有，而不是在功能页面里被临时拼装。

布局选择、删减与显隐策略

不同产品可以选择不同的结构子集：有的产品完全移除 Menu Bar，有的把 Top Tool Tray 合并进 Top Tool Bar，有的默认折叠 Right Panel，有的从不暴露 Bottom Panel。这些选择不会改变架构原则，只是在同一套布局语法里选用不同子集。



图 6.2: 均衡工作台布局: 从完整结构语法中选取较小子集

表 6.8: 布局区域策略: 核心必需与可选/可切换表面

区域类别	典型策略	架构解读
Title Bar + Main Area	核心必需	没有它们，壳层失去身份与主要内容承载能力
Menu Bar / Tool Bars / Tool Tray	产品可选	是否暴露取决于工作流密度与命令表面策略
Left / Right / Bottom Panels	可切换容器	可根据用户偏好或当前任务显示、隐藏、缩放、折叠
Sub Panels	由上下文派生	仅当父面板需要进一步结构分割时才存在

这正是 Layout 与 Page 的边界开始真正生效的地方。某个 Page 不应决定应用是否存在 Bottom Panel，也不应自行决定 Left Panel 是否折叠。这些都是壳层级布局策略，即便其状态会受到当前任务或偏好影响。

覆盖层表面仍属于同一布局家族

带有覆盖层的轮廓图表达了第二个结构观点：弹窗、对话框与类似的临时表面，并不意味着底层布局模型发生了变化。它们只是叠加在同一壳层与容器系统之上。

壳层仍然拥有主空间语法。覆盖层只是暂时占据注意力，并不重定义宿主布局本身。守住这一点，才能避免临时交互表面意外夺走应用永久结构的所有权。

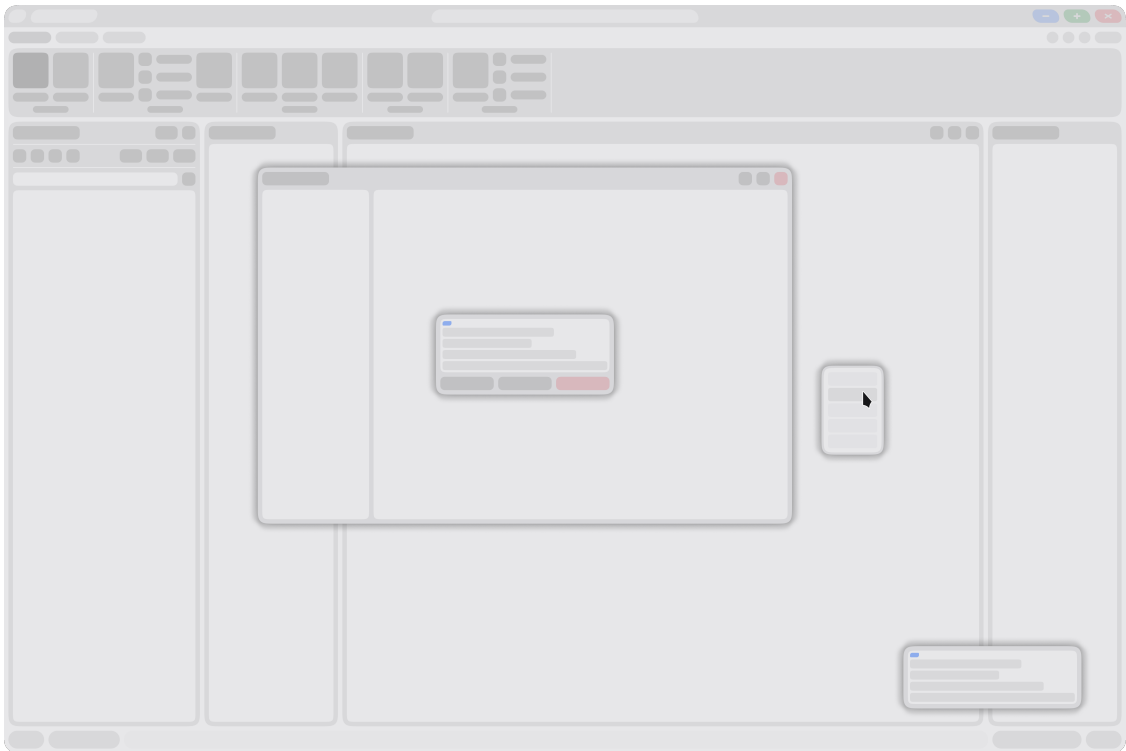


图 6.3: 覆盖层布局：在同一工作台壳层之上的弹窗与对话框表面

视图路由把壳层连接到内容

视图路由子系统负责把静态壳层转变为可导航的运行时表面。它是壳层承载 Page 与复合视图的边界，也是防止路由、状态归属与壳层结构塌成一层的关键分离面。

6.6 依赖与放置规则

允许的依赖压力

架构中的依赖压力应从壳层宿主向下流动到可执行能力，再向下流动到局部视图粒子。

表 6.9: 依赖压力与放置逻辑

层级	拥有	不应拥有
Layers / shells	宿主表面、结构槽位、启动挂钩	与壳层归属无关的产品 workflows 或特权策略
Functions	运行时能力、编排、领域行为、持久化访问	直接的壳层几何结构或过度原子化的渲染细节
Elements	局部渲染与局部信号发射	业务解释、特权资源策略或跨界面编排

放置规则

以下规则概括了本章所要求的结构纪律：

- 1. 窗口壳层应保持为结构宿主，而不是业务逻辑容器

2. 跨切面的运行时能力应位于 `functions/system/`，而不是随意散落在视图文件中
3. 产品特定工作流应在 `functions/business/` 中拥有专属归属地
4. `Elements` 应保持可复用、上下文负担轻、与业务无关
5. 支撑材料、日志、计划、参考资料与维护脚本必须留在运行时代码路径之外

常见架构违例

最常见的退化包括：把业务流程塞进窗口 HTML 或壳层脚本；把特权资源控制藏进页面组件；把系统级配置读取逻辑散进多个局部视图；让 `elements/` 承担跨界面编排；或者把开发支撑脚本混入主运行路径。这些都在模糊所有权，使结构重新失去解释力。

6.7 与相邻章节的关系

本章不是孤立存在的目录规范，而是前后章节的结构收束点。[第 2 章](#) 给出视图实体词汇；[第 5 章](#) 给出模式依据；[第 7 章](#)、[第 8 章](#) 与 [第 9 章](#) 则分别定义交互、状态与资源控制的责任边界。本章把这些约束落入项目结构，使它们拥有可维护的物理形态。

第 7 章

交互系统 与 命令架构

本章是 [第 2 章](#) 与 [第 5 章](#) 的工程补篇。第 2 章定义了 `Command` 相关词汇的语义边界；第 5 章建立了 `Command` 与 `Observer` 的模式基础；本章则规定运行时交互层本身应如何工作：原始输入如何被归一化、命令标识如何被解析、执行如何被派发，以及如何避免业务逻辑重新泄漏进 UI 层。命令执行后的状态持久化与切换，则属于 [第 8 章](#)。

7.1 设计目标与总原则

交互层存在的目的，是把“触发形式”与“执行逻辑”解耦。键盘快捷键、按钮点击、菜单选择、触控手势，都是异质触发源；系统必须把它们统一翻译成命令导向的执行路径。

无逻辑 UI

UI 控件不得承载业务逻辑。诸如 `sys-btn`、`menu-item`、`toggle` 等视图层元素，只是发射器。它们的职责应当收缩为暴露或发射一个 `Command ID`。

输入多态

不同输入来源可以表达同一用户意图，因此内核应识别 `Command ID`，而不是依赖具体的物理触发源。

原子能力

每一项可执行用户能力都应表示为一个原子命令单元，例如 `file:save`、`edit:copy`、`view:toggle-sidebar`。命令是交互层中最小、可独立寻址的执行单位。

显式解耦

交互层必须显式分离以下关注点：

- 输入捕获
- 输入到命令的翻译
- 命令查找与派发
- 业务执行

任何把这些关注点直接压缩成 UI 回调的设计，都应视为架构违规。

7.2 逻辑架构

核心三元组

交互架构围绕三个主要运行时元素组织。

表 7.1: 交互层的核心三元组

元素	架构位置	主要职责
Command Registry	内核层	维护 <code>commandID</code> 到命令实例的映射，并负责派发执行
Keybinding Service	输入翻译层	监听物理按键输入，并将其翻译为语义化 <code>Command ID</code>
Smart Emitters	UI 控制层	在视图层声明 <code>Command ID</code> ，并把激活行为送入交互流水线

这三者的分工必须保持稳定：`registry` 负责执行派发，`keybinding service` 负责翻译，`emitters` 只负责意图发射。

交互流水线

交互系统应被理解为一组流水线，而不是一组零散事件处理器。

表 7.2: 从输入到执行的交互流水线

阶段	含义
Input Surface	键盘、鼠标、触控、菜单、命令面板等入口表面捕获原始动作
Trigger Translator	把原始输入翻译成语义化 <code>Command ID</code>
Scope Resolution	在当前激活作用域下选出有效绑定
Command Registry	解析命令实例并派发执行
<code>Command.execute(context)</code>	作为统一运行时入口执行选中的命令
Service Layer	在 UI 层之外完成业务工作
State / Feedback Layer	在执行后施加状态迁移并生成反馈

UI 层在发射命令时就应停止，不得绕过交互层直接调用 `service`。

作用域解析

作用域感知是交互层的必备子系统。没有作用域解析，同一物理输入会在对话框、编辑器、容器与全局上下文之间产生歧义或冲突。

推荐的查找优先级为：

1. 当前激活的 `modal` 或 `dialog` 作用域
2. 聚焦的 `editor` 或局部 `widget` 作用域

3. container 或 page 作用域

4. 全局 application 作用域

最近的活动作用域应获胜；若本地没有绑定，再按确定顺序向外回退。

7.3 命令协议

所有可执行命令都应继承统一基础契约，以便运行时能够一视同仁地对待它们。

基础接口

代码 7.1: 基础命令接口

```
BaseCommand

1 abstract class BaseCommand {
2   id = '';           // e.g. 'file:save', 'view:toggle-sidebar'
3   label = '';        // display label or i18n key
4   icon = '';         // icon token or asset id
5
6   async execute(context) {
7     throw new Error('Method not implemented.');
8   }
9 }
```

这个契约至少提供三项保证：

1. 每个命令都有全局唯一标识符
2. 每个命令都能提供面向 UI 的元数据
3. 每个命令都暴露统一执行入口

在产品需要时，可以继续扩展 `isEnabled(context)`、`isVisible(context)`、`undo(context)` 或遥测元数据，但这些扩展不应破坏统一入口。

Command ID 约定

Command ID 应遵循 [第 1 章](#) 已定义的命名规则：

`module:verb[-modifier]`

例如 `file:save`、`file:save-as`、`edit:copy`、`view:zoom-in`、`data:sync`。像 `save`、`open` 这样的单词命令被禁止，因为它们缺少足够的语义精度。

执行数据流

命令是无状态执行单元，运行时数据应通过 `context` 注入。有效的数据流是：UI 触发 → Command ID → `Registry.resolve(id)` → `Command.execute(context)` → service 执行。

这样的设计使日志记录、排队、宏录制以及未来扩展执行模型都更一致。

Command Context 是执行封套

command context 不是随手塞参数的袋子，而是一次命令调用的执行封套。命令之所以“无状态”，不是因为什么都不知道，而是因为与本次调用有关的运行时事实都从外部通过 `context` 提供。

交互层应在派发时组装这个封套；命令读取它、完成意图，然后返回，不保留可变会话状态。

表 7.3: 命令上下文的典型层次

层次	典型字段	用途
交互事实	commandId, triggerSource, activeScope	描述这次调用如何进入交互层
目标事实	activeEditor, selectedFile, currentSelection	描述命令作用的对象、文档或 UI 焦点
能力访问	services, eventBus, permissions	暴露业务执行所需的运行时能力
执行元数据	timestamp, correlationId, locale	支持追踪、遥测与环境敏感行为

这也明确了一条边界：**context 是执行快照，不是状态容器**。持久状态、恢复规则与跨命令状态迁移属于 [第 8 章](#)；context 只为当前调用提供必要事实。

上下文边界规则

为了保持有用性，command context 应遵守以下规则：

1. **延后组装**。在派发时创建 context，以反映最新作用域、选择与环境。
2. **只携带必要信息**。只包含当前命令所需的最小充分事实。
3. **不要把 context 当持久状态**。长寿命 workflow 数据、编辑历史与可恢复 UI 状态不属于命令封套。
4. **优先传递抽象而非原始 UI 对象**。能传 service facade、已选实体或 editor adapter 时，就不应传大而可变的视图对象。
5. **命令外部拥有上下文**。交互层负责构造，命令只消费，不得私藏为会话状态。

示例：上下文组装

下表展示了 `file:save` 在派发时可能形成的一组上下文字段。重点不在于固定字段名，而在于把模型明确下来。

表 7.4: 派发时命令上下文示例

字段	示例值	执行中的角色
<code>commandId</code>	<code>file:save</code>	标识当前执行的用户意图
<code>triggerSource</code>	<code>keyboard</code>	记录触发来源
<code>activeScope</code>	<code>editor</code>	记录本次绑定解析使用的作用域
<code>activeEditor</code>	<code>active editor handle</code>	暴露命令作用的主要目标对象
<code>currentSelection</code>	当前选择快照	提供执行所需的局部运行时事实
<code>services</code>	文件服务与通知服务	提供完成操作所需的能力
<code>correlationId</code>	<code>op-20260513-0142</code>	支持跨层日志关联与追踪
<code>timestamp</code>	当前派发时间	记录此次调用何时组装
<code>locale</code>	<code>zh-CN</code>	支持本地化标签、格式与反馈

这个例子刻意保持节制。重点是：命令应当对一个有边界的调用上下文执行，而不是向外抓取隐形全局状态。

7.4 输入映射与上下文路由

一个意图，多种触发

同一命令应能从多个输入表面被调用。

表 7.5: 输入多态：同一意图跨多种触发进入系统

输入表面	示例	最终命令 ID
Keyboard	<code>Ctrl+S</code>	<code>file:save</code>
Toolbar button	点击保存按钮	<code>file:save</code>
Menu item	File → Save	<code>file:save</code>
Command palette	“Save File”	<code>file:save</code>
Touch gesture	点击保存图标	<code>file:save</code>

这就是输入多态在系统层面的具体含义。

作用域感知绑定

同一个物理输入，在不同作用域下可能合理映射到不同命令。

因此，交互层必须把“物理输入映射”与“语义命令派发”明确分离。

区分 Scope 与 Context

Scope 与 context 不是同一概念：

表 7.6: 同一物理输入在不同作用域下的绑定差异

作用域	绑定示例
Global Scope	Ctrl+N → 新建窗口
Editor Scope	Ctrl+N → 光标下移一行
Dialog Scope	Enter → 确认当前操作

- *Scope* 决定选中哪条绑定。
- *Context* 提供被选中命令执行所需的运行时数据。

例如: *scope* 决定对话框中的 Enter 解析为 `dialog:confirm`; *context* 则提供当前对话框实例、焦点字段与校验状态。

7.5 智能发射器与快捷键服务

智能发射器

UI 控件应声明一个 Command ID，并在激活时发射它。

代码 7.2: 发射器侧的命令声明

HTML emitter markup

```
1 <sys-btn command="file:print" label="Print Doc"></sys-btn>
2 <menu-item command="file:print"></menu-item>
```

发射器不应知道最终由哪个 `service` 执行操作，也不应知道执行发生在 `renderer` 还是 `main` 侧，或同一命令是否同时暴露给快捷键、菜单与命令面板。

快捷键服务

`keybinding service` 的职责，是把物理键盘输入翻译成语义化 Command ID。

代码 7.3: 快捷键到命令的翻译

KEYMAP_DEFAULT

```
1 const KEYMAP_DEFAULT = {
2   'Ctrl+P': 'file:print',
3   'Cmd+P': 'file:print'
4 };
```

它负责翻译与派发起点，而不负责直接执行业务。

7.6 实现指南

示例：添加 `file:print`

下面示例展示了为交互层引入一个新命令的最小工程路径。

代码 7.4: 打印命令实现

```
print.command.js
1 export class PrintCommand extends BaseCommand {
2   constructor() {
3     super();
4     this.id = 'file:print';
5     this.label = 'menu.file.print';
6     this.icon = 'print-icon';
7   }
8
9   async execute(context) {
10    window.print();
11  }
12 }
```

代码 7.5: 注册并暴露打印命令

```
Registration and bindings
1 commandRegistry.register(new PrintCommand());
2
3 export const KEYMAP_DEFAULT = {
4   'Ctrl+P': 'file:print',
5   'Cmd+P': 'file:print'
6 };
7
8 <sys-btn command="file:print"></sys-btn>
9 <menu-item command="file:print"></menu-item>
```

命令只定义一次，但可以被一致地暴露到多个交互表面。

7.7 工程约束与禁令

必须遵守

1. 命令必须保持无状态，运行时状态从 `context` 读取
2. UI 层必须发射命令标识，而不是直接发起业务调用

3. 命令标识必须语义精确，并带命名空间
4. `enablement` 应从运行时 `context` 推导，而不是写死在模板中
5. 作用域解析必须显式、确定且可检查

严格禁止

1. 直接在 UI 控件内部嵌入业务逻辑
2. 把可变会话状态存进命令对象
3. 通过匿名回调绕过 `registry`，形成隐藏执行路径
4. 当职责属于响应式 UI 层时，却在命令实现里做直接 DOM 操作

7.8 与相邻章节的关系

本章必须与相邻关注点保持清晰边界：

- 第 1 章 规定了 `module:verb[-modifier]` 这类标识符的书写规则。
- 第 2 章 定义了命令相关术语的语义边界。
- 第 5 章 解释了为什么 *Command* 与 *Observer* 是此处的合适基础模式。
- 第 8 章 负责命令执行后的状态保存、恢复与迁移。

第 7 章回答的是：意图如何进入系统。第 8 章回答的是：该意图执行后，系统状态如何变化并被正确保存。

7.9 可借鉴的参考思想

研究成熟系统的意义不在于历史威望，而在于许多稳定的交互设计思想已经收敛，可直接移植到本章。

实际教训是：交互层不应只负责“执行命令”，它还必须为意图、路由、可用性与跨入口复用建立稳定抽象。

表 7.7: 成熟交互系统中可借鉴的设计思想

来源	可借鉴思想	在本系统中的推荐用法
GoF Command	把执行封装成具有稳定入口的对象	让每项用户能力都服从统一 <code>execute(context)</code> 契约
Cocoa Target-Action	让控件发射动作，而不是承载业务行为	让按钮、菜单项等 UI 元素只充当发射器
Qt QAction	用单一动作定义集中管理标签、图标、快捷键、可用性与调用	把命令元数据作为可复用动作描述，在菜单、工具栏与未来命令面板间共享
Emacs keymaps	让同一物理输入在不同活动作用域下有不同解释	把作用域解析做成显式、分层、有顺序的机制
VS Code Commands API	把命令注册、可发现性与快捷键绑定从 UI 控件里拆出来	把 <code>registry</code> 、 <code>keybinding</code> 与 <code>discoverability</code> 视为一等子系统

第 8 章

状态管理 与 核心服务

本章是 [第 7 章](#) 的工程续篇。第 7 章定义用户意图如何经由命令、快捷键与发射器进入系统；本章则定义这些命令执行之后，系统如何保存、恢复、校验并迁移运行时状态。重点不在“做一个大 Store”，而在服务层所有权、文件生命周期、上下文恢复、配置分层与系统反馈的治理。

8.1 设计目标与总原则

状态管理层的目的，不是把一切变化中的值都塞进某个全局容器，而是确保运行时状态拥有清晰拥有者、合法生命周期，以及可恢复的表示形式。

逻辑必须脱离具体 UI

业务服务不应依赖具体控件。文件 I/O、配置管理、视图状态持久化与反馈编排属于逻辑层，而不是按钮、页面或编辑器组件。

切换必须视为事务

从文件 A 切到文件 B，再返回文件 A，并不是简单地改一下指针，而是由状态捕获、源切换与状态恢复组成的一次事务。只做中间步骤的设计，最终一定会丢失用户上下文。

状态不是倾倒塌

系统应仅保存保证正确性与恢复性所必需的状态。派生值在条件允许时应重算，而不是盲目缓存。

所有权必须显式

每一类重要状态都必须有清晰拥有者：

- 文件生命周期状态属于 *file service*
- 视图恢复快照属于 *context state manager*
- 偏好值属于 *configuration service*
- 瞬时系统通知属于 *feedback service*

一旦所有权含混，调试与恢复都会失去可靠性。

8.2 服务内核

服务是运行时单元

主要逻辑能力应表现为由应用内核管理的 **service**。**service** 是一个长生命周期运行时单元，具有显式的初始化、释放与变更通知职责。

代码 8.1: 基础服务契约

```
BaseService
1 class BaseService extends EventTarget {
2   async init() {}
3
4   dispose() {}
5
6   emitChange(key, value) {
7     this.dispatchEvent(new CustomEvent('change', {
8       detail: { key, value }
9     }));
10  }
11 }
```

这个基础契约带来三项保证：

1. 内核可以按已知顺序初始化服务
2. 内核可以在关闭或重启时可预测地释放服务
3. 依赖层可以观察有意义的状态变化，而不产生直接耦合

为什么服务必须由内核拥有

如果 **service** 由视图随意实例化，很快就会出现三类故障：生命周期碎裂、状态所有权重复、恢复与同步行为不可判定。因此，**service kernel** 是命令执行与持久应用行为之间的稳定边界。

8.3 文件系统服务

虚拟文件系统抽象

文件层不应假定存储永远是本地磁盘。合格的 **file service** 应以抽象文件系统契约为起点，让本地文件、数据库文档与云端内容都能通过同一逻辑 **API** 被处理。

这一抽象的价值不在“纯洁”，而在于存储拓扑改变时，编辑器层不需要跟着一起改写。

表 8.1: 典型的虚拟文件系统操作

操作	用途
read(path)	读取文件或文档的持久表示
write(path, content)	将当前表示写回存储后端
delete(path)	通过存储抽象删除资源
watch(path)	侦测外部变化并反馈到运行时状态模型

文件生命周期应视作状态机

系统不应把文件状态压缩成“已保存/未保存”一个布尔值。真实编辑器 workflow 依赖更丰富的生命周期模型。

表 8.2: 文件生命周期状态及其要求

标志 / 状态	含义	触发条件	所需行为
isDirty	内存内容与持久存储不一致	用户修改内容	打开保存能力, 并阻止不安全关闭
isVirtual	文档仅存在于内存中	新建文件但尚无路径	首次持久写入必须走 Save As
isOrphaned	后端资源已被外部移除	文件监听发现删除/断链	阻止覆盖保存并引导用户恢复
isReadOnly	当前能力无法写入文档	权限限制或后端策略	进入只读模式并明确暴露限制

它们可以在数据模型中表示为标志位，但工程思维仍应是状态机式的：迁移有意义，非法迁移必须被拒绝，行为由当前状态与输入事件共同决定。

8.4 上下文状态管理器

它解决的问题

当用户从文件 A 切到文件 B，再返回文件 A 时，系统应恢复之前的工作上下文：滚动位置、选择区、局部视口状态以及文件级 UI 记忆。没有这一层，切换就会破坏连续性。

视图状态快照模式

可恢复状态应被显式表达为快照对象。重点不在具体字段，而在于“可恢复状态必须被正规化”，而不是隐含在活动 DOM 或视图实例里。

切换是三阶段事务

上下文切换应被建模为三阶段事务：

这个模型能防止两种常见失误：在当前编辑器尚未稳定时过早捕获；在目标视图尚未准备好时过早恢复。

表 8.3: 视图状态快照中的典型字段

字段	示例值	恢复中的作用
lastActiveTime	最近时间戳	帮助排序近期工作上下文
scrollTopPosition	视口坐标	恢复长文档的可见区域
selection	anchor/head 范围	恢复编辑焦点与光标语义
camera	缩放与中心坐标	恢复白板或画布类状态
activeSidePanel	outline	恢复影响工作连续性的局部 UI 选择
localSearchQuery	TOD0	恢复文件局部工作上下文，而非全局搜索

表 8.4: 上下文切换的三阶段事务

阶段	职责
Dehydration	当前编辑器导出其可恢复状态快照
Swap	活动文档或路由切换到目标资源
Rehydration	目标编辑器在渲染就绪后恢复已保存快照

可恢复状态不等于持久业务状态

context state manager 应保存的是工作会话恢复所必需的信息，而不是系统中的所有领域事实。光标位置、相机视口、局部搜索词属于这里；长期项目元数据和业务实体则不属于这里。

8.5 配置服务

配置是一组经过校验的数据

配置值不是杂项键值对，而是产品契约的一部分，因此必须具备校验、优先级规则与可预测运行行为。

核心职责包括：

- *schema* 校验
- 在安全范围内支持热更新
- 跨作用域持久化
- 向依赖子系统广播变更

分层配置优先级

一个实用的设置系统至少应支持如下优先级：

default < user < workspace

这使产品能够提供安全默认值、尊重用户偏好，并在协作场景下允许项目级覆盖。

热更新与安全边界

并非所有设置都应支持即时生效。主题、字号、面板显隐与编辑行为常可立即更新；而安全敏感执行路径、外部二进制路径或存储后端，则往往需要更严格的范围限制与重启边界。

8.6 反馈服务

统一系统反馈

成熟系统不应把提醒、Toast、Banner 与控制台消息散落在无关组件中。面向用户的系统反馈需要一个逻辑拥有者，以便优先级、去重、时序与消失策略保持一致。

消息严重级策略

表 8.5: 反馈严重级策略

级别	典型含义	典型行为
Error	操作失败或用户动作无法继续	需要强可见性，往往要求显式关闭
Warning	风险、降级或外部不一致存在	可持续显示，直到用户察觉或处理
Success	请求动作成功完成	通常短暂显示并自动消失
Info	非阻塞系统进度或环境状态	更适合进入状态区域，而非打断式弹层

Store 与 Renderer 的分离

决定“哪些反馈存在”的逻辑，应与“如何渲染反馈”的 UI 分离：

- **FeedbackStore** 拥有排队、优先级、去重与生命周期。
- **FeedbackRenderer** 订阅状态变化并渲染视觉形式。

这样能让反馈策略保持可测试，并独立于当前 UI 呈现风格。

8.7 实现指南

让编辑器具备可恢复能力

每一种编辑器式视图都应显式提供两个能力：导出当前可恢复状态快照，以及在渲染就绪后恢复先前快照。

这里隐藏的关键约束是顺序：恢复必须发生在目标编辑器真正可用之后，而不只是路由切换之后。

8.8 工程约束与禁令

代码 8.2: 编辑器快照导出与恢复契约

TextEditor

```
1 class TextEditor extends BaseElement {
2   getSnapshot() {
3     return {
4       scrollTop: this.editor.scrollTop,
5       selection: this.editor.getSelection()
6     };
7   }
8
9   async restoreSnapshot(state) {
10    if (!state) return;
11
12    await this.waitForRender();
13
14    this.editor.scrollTop = state.scrollTop;
15
16    if (state.selection) {
17      this.editor.setSelection(state.selection);
18    }
19  }
20 }
```

必须遵守

1. 长寿命应用行为应由 *services* 持有，而不是由视图组件持有
2. 状态迁移应依据当前状态校验，而不是无条件写入
3. 可恢复视图状态应显式且在可能时可序列化
4. 配置应服从清晰的优先级与校验边界
5. 反馈应由中心策略统一排序，而不是由无关控件各自发声

严格禁止

1. 把核心业务状态塞进短命 UI 组件
2. 把持久领域状态与临时恢复快照混在一起
3. 让任意控件绕过 *service* 直接写入存储后端
4. 把所有运行时值都视为“全局状态”，却没有所有权区分
5. 把反馈策略与某一种渲染器实现绑死

8.9 与相邻章节的关系

本章必须与相邻关注点保持分离：

- 第 7 章 定义意图如何通过命令、快捷键与发射器进入系统。
- 本章定义 *services* 如何接收、保存、恢复、校验并迁移由这些意图引出的状态。

第 7 章回答：系统如何决定要做什么？

第 8 章回答：这些结果状态应该存放在哪里，以及之后如何被正确恢复？

8.10 可借鉴的参考思想

研究现有系统的价值，不在于历史名望，而在于许多成熟思想可以直接转译到服务与状态设计中。

表 8.6: 状态管理与核心服务中可借鉴的设计思想

来源	可借鉴思想	在本系统中的推荐用法
GoF Memento	在不暴露内部实现细节的前提下捕获并恢复状态	用显式快照处理文件级视图恢复
Statecharts / SCXML lineage	让状态迁移显式、可测试，并受当前状态约束	把文件生命周期与切换流程建模为迁移驱动逻辑，而不只是散乱标志位
VS Code settings model	用清晰优先级与受限覆盖边界管理分层配置	把配置作为经校验、可感知作用域的数据，而不是任意可变全局量
Redux style guidance	保持状态最小化，让复杂逻辑离开组件	防止状态层沦为倾倒塌或 UI 自有副作用区

实际教训是：良好的状态管理不是一个庞大 Store，而是所有权、迁移规则、恢复策略与有边界持久化的纪律化组合。

第 9 章

并发控制 与 资源管理

本章是 [第 7 章](#) 与 [第 8 章](#) 的运行时续篇。第 7 章解释用户意图如何被归一化为命令，第 8 章解释命令执行后的状态如何被保存与恢复；本章则规定特权资源如何被串行化、窗口与进程边界如何协同，以及如何防止重复或重叠操作把桌面运行时推入无效状态。

9.1 设计目标与总原则

桌面应用中的并发控制，不是为了让所有事情都尽可能同时发生，而是为了避免对特权资源的冲突访问，使所有权边界保持显式，并确保窗口、对话框与 IPC 路由在重复触发或重叠触发时仍然具有确定性。

特权能力必须有单一拥有者

窗口创建、原生对话框、应用生命周期钩子与特权 IPC 处理器都应由主进程拥有。这些能力不是普通 UI 关切，不能被当作 `renderer` 本地行为处理。

进程边界就是架构边界

`renderer` 并不是 `main` 的轻量延伸，而是拥有不同信任级、不同生命周期与不同故障模式的独立运行边界。因此，架构必须显式表达进程所有权。

并发控制是选择性的，不是全局性的

并非所有动作都需要串行化。并发控制应重点落在独占、昂贵、安全敏感或经由操作系统中介的资源上，例如原生对话框、窗口生成、关闭协调与特权 IPC 入口。

重复触发必须无害

快速连续输入不应制造重复窗口、重复对话框或重叠破坏性操作。如果同一意图在短时间内多次到达，系统必须根据策略选择聚焦、合并、忽略或串行。

9.2 运行时边界：Main、Renderer 与 Utility

为什么桌面运行时是多进程

基于 Chromium 的桌面运行时天然采用多进程模型，因为把 UI 渲染与应用控制分离，有利于故障隔离、权限收缩与资源边界控制。

表 9.1: 桌面运行时中的进程所有权

进程	主要职责	并发上的理由
Main process	生命周期控制、顶层窗口、原生对话框、特权 IPC	把独占资源与策略决策交给唯一权威拥有者
Renderer process	视图渲染、局部交互、命令发射	防止 UI 触发直接竞争特权原生能力
Utility / child process	重 CPU、易崩溃或需要隔离的工作	将高风险或阻塞性工作移出主事件循环

实际结论很直接：**main** 拥有生命周期与特权资源，**renderer** 拥有表现与局部交互，辅助进程负责隔离高风险工作。把这些责任重新揉成一个大拥有者，只会把多进程架构原本要切断的故障耦合重新接回来。

主进程职责

主进程应拥有应用启动与关闭、窗口创建与销毁、原生对话框与 OS 集成、特权 IPC 路由，以及进程级故障的恢复编排。

渲染进程职责

renderer 应集中于视图渲染、局部交互与命令发射。它可以请求特权工作，但不得自行决定窗口权威，也不得绕开进程边界。

辅助进程是安全阀

如果某任务计算量大、易崩溃或具备独立操作意义，就应尽量移出 **main**。并发控制不仅是“加锁”，也是避免把一切高风险工作塞进同一个事件循环。

9.3 窗口管理模型

窗口注册表是真相来源

多窗口应用需要显式窗口注册表。这个注册表既要按运行时身份识别窗口，也要按语义角色识别窗口。

没有注册表，重复触发很快就会退化成重复窗口与泄漏引用。

表 9.2: 窗口注册表的职责

职责	为什么重要
按语义标签查找	让 <code>settings</code> 、 <code>about</code> 、 <code>help</code> 这类请求可被确定性解析
跟踪父子关系	保持 <code>modal</code> 所有权，避免阻塞流脱离父窗口
关闭时移除	防止窗口关闭后仍残留幽灵引用与错误焦点目标
支持 <code>create-or-focus</code> 决策	让运行时决定复用已存在窗口还是创建新窗口

已有窗口优先聚焦

设置、关于、帮助、偏好等辅助窗口，通常都应采用“已存在则聚焦”的策略：先检查同语义标签窗口是否存在；若存在，则恢复并聚焦；若不存在，再创建并登记。与盲目重建相比，这一策略更适合稳定工具表面。

父子关系与模态关系

当某流程必须阻塞父表面的交互时，应通过父子所有权与 OS 级模态语义来表达，而不是只做一个“看起来居中”的弹窗。阻塞语义必须被编码在窗口管理器里，而不是在 `renderer` 里临时拼接。

生命周期清理

窗口销毁后，必须从注册表移除，并释放与之关联的窗口级资源。很多并发问题之所以长期潜伏，原因正是 UI 看似关闭了，但后台仍保留旧引用。

9.4 独占资源控制

为什么原生对话框需要串行化

原生对话框属于 OS 中介下的独占资源。如果 `renderer` 可以无协调地连续触发同一个 `open-file` 或 `save-as`，对系统而言就可能出现重复对话框、焦点冻结或资源争抢。

这正是一个“小型互斥”能换来巨大稳定性的场景。

对话框守卫契约

`dialog guard` 应至少提供一种能力：在某个语义 `key` 下独占地运行任务。

设计重点不是类名，而是“独占性必须被集中、可观察地表达出来”。

锁键设计

锁键应当同时体现资源范围与业务动作，例如：

- `win-12:native-dialog`

代码 9.1: 原生对话框的独占守卫

DialogGuard

```
1 class DialogGuard {
2     constructor() {
3         this.locks = new Set();
4     }
5
6     async runExclusive(key, task) {
7         if (this.locks.has(key)) return null;
8
9         try {
10             this.locks.add(key);
11             return await task();
12         } finally {
13             this.locks.delete(key);
14         }
15     }
16 }
```

- `main:open-project`
- `settings:save-preferences`

过粗的键会串行化本不相关的工作，过细的键又无法阻止重复。锁空间应当精确投影真实独占边界。

忽略、合并或排队

不同重复请求不应被同一种策略处理。

表 9.3: 重复或重叠请求的推荐策略

资源类型	推荐策略	原因
原生文件对话框	丢弃重复请求	用户需要一个选择器，而不是多个重叠系统对话框
单例辅助窗口	合并为 focus-existing	正确目标已存在，应该复用而不是重建
同一文档的保存操作	串行或合并	持久写入不应竞争或乱序完成
应用关闭	全局加锁	可重入关闭会制造不一致的注册表与生命周期状态
遥测或后台刷新	在有意义时排队	顺序完成本身仍可能具备语义价值

对于原生对话框与单例窗口，默认策略应是 `ignore-or-focus`，而不是无界排队。

9.5 IPC 路由与校验

共享通道契约

IPC 通道名必须作为共享契约被集中定义，而不是散落成各个 `renderer` 与 `main handler` 中的字符串字面量。

代表性例子包括：

- `window: minimize`
- `window: open-settings`
- `dialog: open-file`
- `app: request-quit`

这样做能让路由可检查，并减少通道语义漂移。

主进程集中路由

主进程应集中注册 IPC handler，以便权限检查、`sender` 校验与生命周期策略能够在同一处被看见。

代码 9.2: 单例辅助窗口的主进程 IPC 路由

ipcMain.handle

```
1 ipcMain.handle('window:open-settings', async (event) => {
2   if (!validateSender(event.senderFrame)) return null;
3   return windowManager.createOrFocus('settings');
4 });
```

sender 校验是强制项

IPC 不是因为来源于“应用内部”就天然可信。只要特权工作通过 IPC 暴露，就必须在执行动作或返回敏感数据之前校验 `sender`。这既是安全要求，也是并发要求：未经授权或重复来源不应争夺主进程特权能力。

广播与请求-响应必须区分

IPC 至少应区分两种完全不同的通信形态：

- 带校验、带确定结果的 *request-response*，用于特权操作
- 面向非特权状态更新的 *broadcast / event fan-out*

把这两类模式混入一个模糊 `event bus`，只会让竞态更难分析。

9.6 生命周期协调与故障围堵

单实例协调

如果应用在概念上是单实例，就应显式协调第二次启动。通常的正确行为不是允许多个主实例竞争同一套应用级资源，而是：启动时请求单实例锁；如果失败，则把意图转发给主实例并退出；在主实例里恢复并聚焦相关窗口。

优雅关闭是一笔事务

应用关闭应被视为协同事务，而非原始进程退出。

表 9.4: 优雅关闭的阶段

阶段	职责
Validation	询问脏状态或挂起工作是否阻止关闭
Persistence	允许 <code>renderer</code> 持有的状态先持久化，或在产品要求下否决关闭
Teardown	释放窗口级与进程级资源，并注销全局钩子
Release	清空注册表、释放锁，仅在清理完成后允许最终退出

绕过这些阶段的“立即退出”，只能保留给极端异常情况。

崩溃可见性与恢复边界

当 `renderer` 或 `child process` 崩溃时，应用应在对应拥有边界处响应，而不是默默污染全局假设。最低要求包括：记录故障事件、废弃失效进程或窗口的旧引用、在用户仍有选择权时暴露恢复方案，以及仅通过正常 `manager` 路径重建表面。

9.7 实现指南

Create-or-Focus 窗口策略

`window manager` 应为单例辅助窗口暴露一个显式方法。

这段策略把所有权集中起来，避免重复创建，并把清理附着在同一处窗口生成策略之上。

原生对话框入口守卫

`renderer` 侧去抖只改善体验，但不是架构保证。真正的保证必须位于 `main` 侧的 `guard`，因为只有主进程真正拥有底层原生能力。

9.8 工程约束与禁令

代码 9.3: 单例窗口的 *create-or-focus* 策略**createOrFocus**

```
1 function createOrFocus(tag, factory) {  
2   const existing = registry.get(tag);  
3  
4   if (existing && !existing.isDestroyed()) {  
5     if (existing.isMinimized()) existing.restore();  
6     existing.focus();  
7     return existing;  
8   }  
9  
10  const win = factory();  
11  registry.set(tag, win);  
12  win.on('closed', () => registry.delete(tag));  
13  return win;  
14 }
```

必须遵守

1. 特权窗口与对话框操作必须由主进程拥有
2. 单例辅助窗口必须采用显式 *create-or-focus* 逻辑
3. 独占资源必须以语义锁或等价串行机制保护
4. *IPC handler* 在执行特权动作前必须校验 *sender* 身份
5. 关闭与崩溃处理必须可预测地清理注册表状态并释放引用

严格禁止

1. 在 *renderer* 本地路径里直接创建特权窗口
2. 允许重复触发无限生成原生对话框或辅助窗口
3. 只在 *UI* 组件里保存长寿命窗口身份，而 *main* 侧没有注册表
4. 暴露未做 *sender* 校验的原始特权 *IPC* 端点
5. 在需要优雅关闭时，用绕过清理与状态检查的方式粗暴退出应用

9.9 与相邻章节的关系

本章必须与相邻关注点保持清晰边界：

- 第 7 章 解释意图如何进入运行时。

- 第 8 章 解释状态存放在哪里、如何恢复。
- 本章解释如何在重复、重叠与故障情况下，保证窗口、对话框与进程边界上的资源仍具有确定性。

9.10 可借鉴的参考思想

研究成熟平台的价值不在于历史权威，而在于许多设计已经证明可复用。

表 9.5: 并发控制与资源管理中可借鉴的设计思想

来源	可借鉴思想	在本系统中的推荐用法
Chromium 多进程架构	用进程隔离控制崩溃、权限边界与资源拥有者	让 <code>renderer</code> 故障留在局部，并把 <code>main</code> 所有权视为严格控制边界
Electron 进程模型	在主进程集中窗口生命周期与原生集成	把窗口创建、对话框访问与生命周期编排移出 <code>renderer</code>
Electron 单实例生命周期	第二次启动时聚焦主实例，而不是允许多个应用根竞争	在应用实例层也采用 <code>focus-existing</code> 策略
Electron IPC 安全指导	校验 <code>sender</code> 并限制特权窗口创建或导航	把 IPC 校验视为并发安全的一部分，而不只是安全清单
GoF Registry / Singleton / Mutex	显式表达身份、独占与运行时所有权	用 <code>registry</code> 驱动窗口查找，并用语义 <code>guard</code> 管理单例或 OS 中介资源

实际教训是：桌面软件中的并发控制，不只是线程安全问题，更是所有权边界、独占资源串行化、生命周期确定性与受限进程通信的综合设计。

第 10 章

算法 与 复杂度控制

本章把 [第 5 章](#)、[第 7 章](#)、[第 8 章](#) 与 [第 9 章](#) 中提出的结构性约束，进一步落实为可执行的计算策略。模式告诉我们职责应该放在哪里；交互、状态与并发章节告诉我们系统各自拥有什么；本章则回答这些子系统应当**如何计算**，例如如何遍历层级、如何排序候选项、如何调度竞争工作、如何处理依赖顺序，以及如何避免每次局部变化都触发全量重算。

10.1 设计目标与总原则

工程文档中的算法章节，不是为了复述一本通用算法教材，而是为了识别本系统中反复出现、真正影响运行时表现的计算问题，并为这些问题设定稳定、可预测的处理规则。

算法是运行时契约

在本系统中，算法不是实现层的小技巧，而是运行时契约。作用域解析决定物理输入最终命中哪个命令；优先级调度决定严重错误会不会被低价值提示淹没；依赖排序决定服务与插件能否无死锁启动；增量更新决定拖拽、搜索与布局调整能否保持交互性，而不是退化为反复的全量重算。

先保证正确性，再谈微优化

一个常数项更小但结果错误的算法，不是优化，而是缺陷。工程顺序应当始终是：先得到正确结果，明确边界行为，再控制复杂度落在可接受范围内，最后才考虑常数级提速。

复杂度必须服从交互预算

同一个复杂度在不同运行路径上的工程意义并不相同。启动阶段的一次 $O(n)$ 扫描可能合理，但把同样的扫描放到每次按键、拖拽帧或滚动事件里，就可能直接破坏体验。因此，复杂度讨论必须始终绑定到用户感知延迟与触发频率。

局部变化优先增量更新

当系统已经处于有效状态，且变化只影响一个局部切片时，优先策略应当是更新受影响部分，而不是重算整棵树、整份文档或整个工作台窗口。这一原则同时适用于布局、搜索索引、状态恢复与反馈渲染。

10.2 核心算法问题族

本桌面系统反复出现的算法问题，主要是结构性的，而非纯数值性的。

表 10.1: 系统中的核心算法问题族

问题族	典型场景	工程目标
树遍历与层级寻址	布局树、文件树、视图树、作用域回退链	在不破坏层级语义的前提下快速命中目标节点
搜索、过滤与排序	命令面板、文件查找、列表过滤、候选排序	在可接受延迟内返回稳定且可解释的结果
优先级调度与队列管理	反馈系统、重复触发处理、串行任务	让重要工作优先完成，并控制重复请求
依赖排序与无环装配	服务启动、插件组合、模块装配	推导安全执行顺序，避免循环依赖与隐性死锁
增量计算与局部重建	拖拽、面板显隐、局部刷新、状态恢复	避免全量重算，缩短热点交互路径

这些问题族并不是新的架构域，而是前几章已经划定的架构边界内部，反复出现的计算义务。

10.3 复杂度预算与运行路径

只有把复杂度和具体运行路径、用户感知绑定起来，复杂度分析才真正有用。

表 10.2: 按运行路径划分的复杂度预算

运行路径	预算直觉	偏好的算法倾向
直接交互路径	单次动作应当接近即时响应	优先使用索引查找、缓存命中与局部更新，避免重复全扫描
滚动、拖拽、缩放	持续交互不能伴随大块同步重算	优先采用增量算法、比例状态与事件合并
命令反馈回路	用户应当感知到动作完成，而不是运行时停滞	优先采用确定顺序与显式队列策略
启动与装配	一次性成本可以稍高，但启动必须稳定可预测	优先使用拓扑排序、显式依赖图与一次性索引
后台维护任务	可以更关注吞吐，但不能拖慢前台体验	优先使用批处理、分块执行与延后调度

成熟的工程问题不是“这是 $O(n)$ 还是 $O(n \log n)$ ”，而是“这笔代价发生在哪条路径上，会重复多少次，是否与用户的直接交互竞争”。

10.4 树遍历与层级寻址

为何树结构会反复出现

系统中大量数据天然呈树状：视图层级从 **Container** 到 **Element** 明确分层，文件导航具备层级关系，面板组合具备嵌套结构，作用域解析也经常表现为从局部向外的祖先回退。因此，遍

历不是偶发工具，而是持续性的设计议题。

遍历顺序必须服从语义

表 10.3: 按问题语义选择遍历顺序

场景	优先遍历方式	理由
寻找最近祖先规则	深度优先祖先回溯	问题从当前上下文向外收缩，而不是横向扫同层兄弟
收集当前可见节点	广度优先遍历	按层展开更贴近用户可见层级
销毁一个容器及其子树	深度优先后序遍历	先释放子节点，再释放父节点，避免悬挂引用
生成导航文本或树形输出	深度优先前序遍历	更自然地产生自上而下的层级顺序

简单原则是：遍历顺序应当因为问题语义而被选择，而不是因为某个术语更熟悉。

只有遍历还不够

当系统已经具备稳定身份时，纯遍历会变得低效。文件路径、面板 ID、Shell 标签、作用域标签都应被视为显式寻址键。在这种情况下，树结构配合旁路索引，通常比每次从根到叶盲扫更合适。

10.5 搜索、过滤与稳定排序

推荐的计算流水线

面向用户的搜索界面，应采用稳定的处理流水线：

- 1. 归一化原始输入与候选文本
- 2. 过滤候选集
- 3. 对剩余候选打分
- 4. 使用显式的并列打破规则进行排序
- 5. 只在排序完成后再裁剪或分页

这个顺序很重要。先排序后归一化，会放大大小写与别名噪声；先打分后过滤，则会浪费算力在本就应被排除的候选项上。

排序必须稳定

用户看到的排名不仅需要主分数，还需要可预测的并列打破规则。当多个候选分数相同，运行时应当回退到第二语义键，最后再回退到稳定身份键。否则同一个查询可能连续几次得到抖动的结果顺序，逐步损耗用户信任。

重复搜索应依赖索引

如果同类查询会反复出现，系统就不应每次都重新清洗原始数据并做全量扫描。归一化关键词索引、路径到节点的查找表、预计算排序键，往往比在未准备的数据集上套一个复杂评分公式更有价值。

10.6 优先级调度、队列策略与有界历史

为什么 FIFO 常常不够

反馈系统是最直观的例子。严重错误不应排在一串低价值成功提示之后。普通先进先出不能满足这一要求，必须有显式的优先级感知调度。

抢占式优先级队列

抢占式优先级队列能提供三种关键性质：高优先级项可以中断低优先级展示；同优先级项仍保留可解释的顺序；当前显示与历史保留可以分开管理。当“紧急程度”和“可见性”比单纯到达顺序更重要时，这是更合适的模型。

丢弃、合并、排队、串行是不同策略

表 10.4: 重复请求与竞争请求的策略区分

策略	适用场景	目的
丢弃	同一原生对话框被重复触发	在形成操作系统级冲突之前直接拒绝重复请求
合并	已存在某个窗口或工具面板的再次请求	把新请求折叠为 focus-existing 行为
排队	顺序本身具有语义价值的后台工作	保留有意义的执行顺序，而不是假装资源独占
串行	独占资源或持久化写入路径	保证同一临界区同一时刻只有一个执行流进入

这些策略不能被含混地统称为“排队”。它们解决的是不同问题，必须有意识地选择。

有界历史宜使用环形缓冲区

当系统只需要保留最近 N 条记录时，无上界堆积就是软弱设计。环形缓冲区为内存设置固定上限，让写入行为保持简单，并把淘汰策略显式化。对于反馈历史、监控快照和其他近期记录视图，这通常是更合理的选择。

10.7 DAG 拓扑与依赖排序

为什么需要拓扑顺序

一旦系统里出现“A 必须先于 B 加载”这样的规则，问题就不再是普通列表排序，而是有向无环图上的依赖排序。服务启动、插件装配与按依赖组装模块，都会落入这一类。

循环检测是必需项

拓扑排序只在无环条件下成立。因此，一个不检测环的依赖装配系统是不完整的。出现循环时，运行时应当显式失败、指出循环路径，并停止启动，而不是把结构错误放任成随机的部分初始化。

启动顺序应被声明为数据

当启动已经成为一个拓扑问题，就不应再把顺序藏在冗长的 `if-else` 或脆弱的手工 `import` 顺序里。更合理的设计是把依赖关系声明成数据，再由一次拓扑遍历计算最终执行顺序。

10.8 增量计算与局部重建

脏标记与版本戳

当系统只改动了一小部分，与其全量重扫，不如标记发生变化的片段、使相关缓存失效，并仅重算受影响的区域。脏标记与版本戳虽然简单，却是高价值的增量工具。

比例状态往往比绝对值更稳定

若布局本质上依赖容器尺寸，保存绝对像素值通常会在后续 `resize` 或重新装配时制造不必要的重算压力。比例状态、相对偏移和标准化的父子关系，在恢复时通常更稳健。

局部重建优于全局失效

隐藏一个面板，不应要求整个工作台全量重建；修改一个节点，也不应触发与其无关的全局重排序。当所有权边界清晰时，优先策略应当是局部差异计算、局部重排、局部快照恢复与局部索引失效，而不是系统级重置。

10.9 算法策略需要升级的信号

以下症状通常说明系统需要的是更强的算法策略，而不是继续堆条件分支：

1. 同一类查询形态下，数据被反复完整重扫
2. 相同用户动作随着数据量增长出现明显延迟分化
3. 等价查询的结果顺序持续抖动
4. 启动顺序依赖人工维护，并会因小改动失效

5. 局部状态变化引发大面积无关重算
6. 近期历史记录无限增长，没有明确上限
7. 面对重复请求，有时要丢弃、有时要合并、有时要串行，但运行时没有显式策略模型

一旦出现这些信号，正确回应往往是重设算法层面的结构，而不是再补一层枝节判断。

第 11 章

版本控制 与 发布工程

本章是 [第 2 章第 7 节](#) 的工程补篇。第 2 章回答的是生命周期阶段应该叫什么；本章回答的是这些阶段信息如何被编码进一个机器可解析、人可阅读的版本字符串，以及如何围绕它建立一条清晰的发布流水线。

11.1 版本号混乱的代价

常见场景是：用户报告“Beta 3 有 bug”，但团队内部已经说不清那是三周前哪个提交、哪一次构建、哪一条分支。根本问题在于：版本号本该是一种自描述信息载体，但大多数项目把它只当作递增计数器。

版本号混乱通常表现为三种退化：

1. **数字膨胀**。一周三次构建，PATCH 从 1 跳到 47，但没有人知道每次递增背后发生了什么。
2. **阶段标签失真**。Alpha 被发给真实用户，Beta 被发给 QA，标签与受众错位，导致整个版本体系失去可信度。
3. **构建不可追溯**。手里只有一个 `app-setup.exe`，却说不清它对应哪条分支、哪个提交、哪一次流水线构建。

完整的版本与发布系统，解决的不是“怎么给一个数字”，而是：给定一个版本字符串，开发、测试、用户与 CI 都能独立推断该版本的稳定性承诺、功能边界、目标受众与所处环境。

11.2 基础：Semantic Versioning 2.0.0

Semantic Versioning 2.0.0 (SemVer) 是当前行业最广泛采用的版本格式规范。本系统以 SemVer 作为格式基础，再叠加 [第 2 章](#) 定义的生命周期语义。

核心规则

本系统直接采纳三条铁律：

1. **发布后不可变**。一旦 1.2.0 发布，其二进制内容不可被静默替换，任何修复都必须发布为新版本。
2. **递增即契约**。MINOR 递增时 PATCH 归零；MAJOR 递增时 MINOR 与 PATCH 同时归零。

代码 11.1: SemVer 核心格式

```
MAJOR.MINOR.PATCH

1 MAJOR.MINOR.PATCH
2
3 MAJOR <-- incompatible API changes
4 MINOR <-- backward-compatible feature additions
5 PATCH <-- backward-compatible bug fixes
```

3. **0.y.z** 表示初始开发阶段。在 1.0.0 之前，API 仍不稳定，任何东西都可能改变。

预发布标识

SemVer 允许在 PATCH 后附加预发布标识：

代码 11.2: SemVer 预发布标识语法

```
Pre-release examples

1 1.2.3-alpha.1 <-- 1st pre-release build, alpha phase
2 1.2.3-beta.2 <-- 2nd pre-release build, beta phase
3 1.2.3-rc.1 <-- 1st release candidate build
```

本系统保留 SemVer 的预发布结构，但不直接沿用传统 alpha / beta / rc，而使用第 2 章定义的生命周期词汇。

构建元数据

代码 11.3: 构建元数据语法

```
Build metadata

1 1.2.3+cafebabe <-- build metadata; ignored in precedence
```

构建元数据用于嵌入提交哈希、流水线构建号等机器信息。它不参与版本优先级比较，但在追溯与定位问题时极其关键。

11.3 生命周期阶段：从 SemVer 到通用语言

第 2 章第 7 节 定义了五个生命周期词汇。本节将它们映射为 SemVer 预发布标识，使版本号同时具备格式正确性与语义自说明性。

五阶段版本格式

表 11.1: 生命周期阶段到 SemVer 的映射（格式与受众）

阶段	SemVer 格式	受众
Experimental	x.y.z-experimental.N	核心开发者
Development	x.y.z-dev.N	内部开发者
Integration	x.y.z-integration.N	QA 团队
Preview	x.y.z-preview.N	公测用户
Candidate	x.y.z-candidate.N	所有发布参与者

表 11.2: 生命周期阶段到 SemVer 的映射（约束与示例）

阶段	约束	示例
Experimental	API 不稳定；不可发给 QA	0.1.0-experimental.3
Development	功能可运行但未系统验证	1.2.0-dev.7
Integration	功能冻结，进入集成验证	1.2.0-integration.2
Preview	功能完整，接受早期用户反馈	1.2.0-preview.1
Candidate	代码锁定，接近正式版	1.2.0-candidate.1

这些阶段词能够直接告诉读者版本对应的目标受众与使用约束，而不需要额外翻译层。

阶段词汇的自说明性

本系统选择 -dev、-integration、-preview、-candidate，而不是无差别沿用传统 alpha / beta / rc，原因在于它们更直接、更少歧义，尤其面对非技术受众与跨语言团队时更友好。

重点

自说明标识的原则是：读者不应先掌握某个圈层缩写，才能理解一个版本字符串。

阶段推进规则

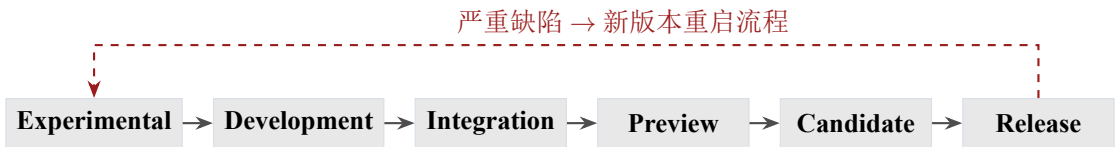


图 11.1: 阶段推进与回退

向前推进是不可逆的。若 Candidate 发现严重问题，不应“修补同一 Candidate”，而应提升版本号并重新开始相应阶段。这正是“发布内容不可变”的直接推论。

11.4 构建产物命名

把 第 1 章 的命名规则应用到版本化构建产物上，可得到以下形式：

代码 11.4: 构建产物命名约定

Artifact names

```
1 # Chapter 1 delimiter rules:
2 #  __  subject (the executable filename)
3 #  _   attributes (version, architecture, platform)
4
5 project-name__1.2.3-candidate.1__x64.exe
6 project-name__1.2.3__x64.dmg
7 project-name__1.2.3__linux.AppImage
8
9 # Source code tarball
10 project-name__1.2.3-candidate.1__src.tar.gz
```

双下划线负责区分主体与属性，连字符负责连接属性内部词语，这与第 1 章定义的层级分隔体系完全一致。

11.5 更新日志约定

更新日志建议基于 Keep a Changelog 规范，并采用统一的变更分类词汇。

分类词汇

表 11.3: 更新日志分类词汇

分类	含义	对版本的影响
Added	新增功能	视 API 边界决定 PATCH 或 MINOR
Changed	既有行为变化	视兼容性决定 MINOR 或 MAJOR
Deprecated	即将移除的功能	MINOR 标记，后续 MAJOR 移除
Removed	已经移除的功能	MAJOR
Fixed	缺陷修复	PATCH
Security	安全漏洞修复	视范围而定，但应优先快速发补丁

格式规范

应遵守五项原则：最新版本写在最前；每个版本都有日期；同类变更归类呈现；从用户视角叙述；未定版号的改动统一放在 [Unreleased] 下。

11.6 发布流水线

版本号决策流程

每次构建前，都应按顺序回答三件事：有没有破坏性变更？有没有新的公开能力？如果都没有，这次是不是仅修复或内部改进？

代码 11.5: 更新日志格式示例

CHANGELOG.md

```
1  ## [1.2.0] - 2026-01-15
2
3  ### Added
4  - New PDF export functionality
5  - Auto dark theme switching
6
7  ### Fixed
8  - Resolved occasional startup crash
9
10 ### Changed
11 - Switched default font from Segoe UI to Inter
```

分支到环境的映射

表 11.4: 分支、阶段语义与环境映射

分支	语义阶段	环境
feature/*	Development	Dev
main（开发合并后）	Integration	QA
release/*	Preview / Candidate	Staging
main（打 tag）	Release	Production

重点

分支名不等于版本号。分支是 Git 导航工具；版本号是制品身份标记。它们唯一的联系，是 CI 可以根据分支默认推断阶段词。

11.7 制品分发决策树

不同阶段词天然对应不同分发范围：Experimental 仅限内部暂存；Development 进入开发通道；Integration 进入 QA；Preview 面向公测；Candidate 面向最终确认；正式 Release 才进入生产用户。阶段词本身就携带了分发范围，不需要再为同一事实额外发明一套标志位。

11.8 版本号工具化实现

代码中的版本号必须服从**单一事实源**原则。推荐做法是在中心数据源中声明版本，再由构建脚本注入运行时。

原则很简单：版本号必须由数据程序化组装，而不是手写拼凑。手写版本字符串极易引入错字、格式违规与事实漂移。

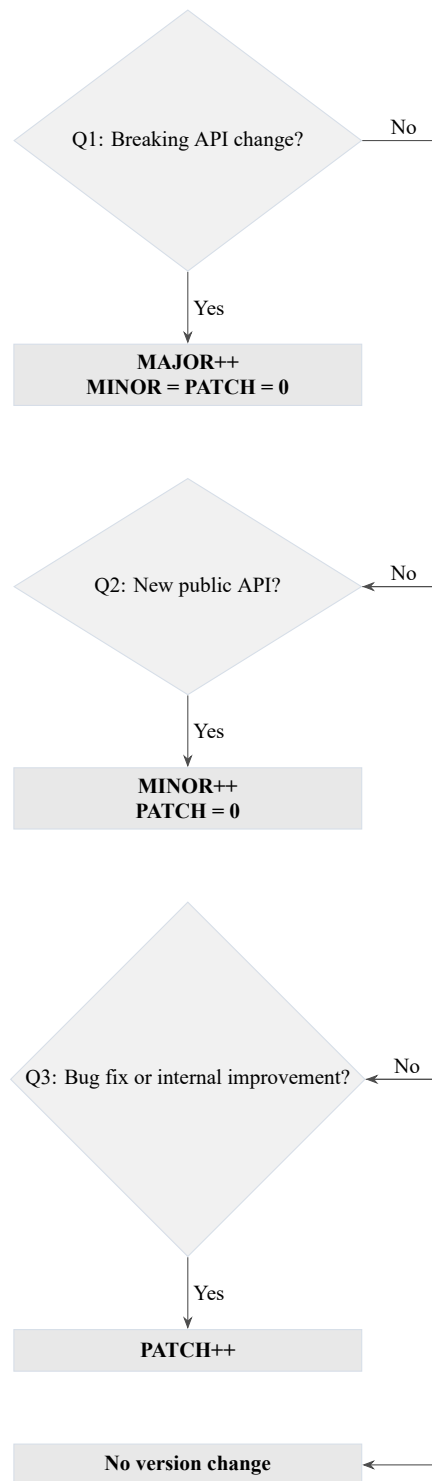


图 11.2: 版本号决策树

11.9 维护原则

多版本分支下的热修复策略

若 1.2.0 发现安全漏洞，而主线已经进入 2.0.0-dev，则正确做法不是在主线上直接打 1.2.1，而是从 1.2.0 的 tag 开出 `release/1.2.x`，修复后发布 1.2.1，再把同一修复合并回主线。

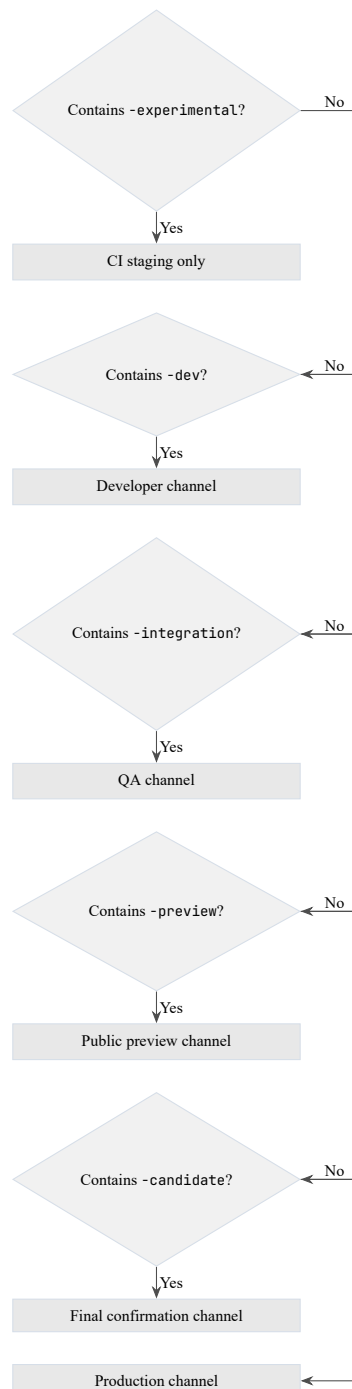


图 11.3: 制品分发决策树

版本号绝不复用

如果 1.2.5 因严重缺陷被撤回，该数字也必须永久退役。下一个修复版本应是 1.2.6，而不是静默重发 1.2.5。

破窗警告

静默替换已发布二进制、无语义地跳号、让 changelog 长期滞后，都会迅速腐蚀整个发布系统的信任基础。版本系统一旦失去可解释性，回滚、排错与协作都会变得昂贵。

代码 11.6: 版本数据源与组装方式

version.json + version.ts

```
1 // version.json — the sole version data source
2 {
3   "major": 1,
4   "minor": 2,
5   "patch": 0,
6   "prerelease": "candidate",
7   "prereleaseNumber": 1,
8   "buildMetadata": "a1b2c3d"
9 }
10
11 // version.ts — runtime version assembly
12 function formatVersion(v: VersionData): string {
13   const core = `${v.major}.${v.minor}.${v.patch}`;
14   const pre = v.prerelease
15     ? `-${v.prerelease}.${v.prereleaseNumber}`
16     : '';
17   const meta = v.buildMetadata ? `+${v.buildMetadata}` : '';
18   return `${core}${pre}${meta}`;
19 }
20 // → "1.2.0-candidate.1+a1b2c3d"
```

11.10 回连第 2 章：版本字符串的两层结构

第 2 章 中定义的 Experimental、Development、Integration、Preview、Candidate 属于语义层，回答“这个版本给谁用，处在什么阶段”；本章定义的 SemVer 格式与发布流水线属于工程层，回答“版本字符串怎么写、何时发布到哪个通道”。

表 11.5: 语义层与工程层的对应关系

阶段	第 2 章（语义层）	第 11 章（工程层）
Experimental	仅限内部，API 不稳定，不进 QA	0.y.z-experimental.N
Development	内部开发构建，日常迭代	x.y.z-dev.N
Integration	仅供 QA，功能冻结	x.y.z-integration.N
Preview	面向公测用户，功能完整	x.y.z-preview.N
Candidate	面向最终确认，代码锁定	x.y.z-candidate.N
Release	面向正式用户，稳定发布	x.y.z

一个自说明版本号必须同时满足这两层：既有正确的阶段词，也有正确的格式与流程语义。缺少任一层，版本号都不完整。

提示

第 2 章的通用语言告诉你版本生命中的每个阶段应当叫什么；本章的 SemVer 与发布流程

告诉你这些名称应当如何被编码为机器与人都能理解的字符串。词汇提供语义，格式提供结构，二者缺一不可。

附录 A

参考文献

命名约定

1. **Zakharchenko, A.** (2017). Naming Cheatsheet. *GitHub*.
<https://github.com/kettanaito/naming-cheatsheet>
2. **Google.** Google Style Guides. *GitHub*. <https://google.github.io/styleguide/>
3. **Airbnb.** Airbnb JavaScript Style Guide. *GitHub*.
<https://github.com/airbnb/javascript#naming-conventions>
4. **Boswell, D., & Foucher, T.** (2011). The Art of Readable Code. *O'Reilly Media*.

领域建模与通用语言

1. **Evans, E.** (2003). Domain-Driven Design: Tackling Complexity in the Heart of Software. *Addison-Wesley*.
2. **Vernon, V.** (2013). Implementing Domain-Driven Design. *Addison-Wesley*.
3. **Vernon, V.** (2016). Domain-Driven Design Distilled. *Addison-Wesley*.
4. **Deissenboeck, F., & Pizka, M.** (2006). “Concise and Consistent Naming.” *Software Quality Journal*, 14(3), 261–282.
5. **Høst, E. W., & Østvold, B. M.** (2009). “Debugging Method Names.” In *ECOOP 2009* (pp. 294–317). *Springer*.
6. **Fowler, M.** (1996). Analysis Patterns: Reusable Object Models. *Addison-Wesley*.
7. **ISO 704:2022.** Terminology work — Principles and methods. *International Organization for Standardization*.

色彩与主题设计

1. **W3C.** (2018). Web Content Accessibility Guidelines (WCAG) 2.1.
<https://www.w3.org/TR/WCAG21/>
2. **Myndex Research.** (2019–). APCA: Advanced Perceptual Contrast Algorithm.
<https://www.myndex.com/APCA/>

3. **Ottosson, B.** (2020). OKLCH Color Space. <https://bottosson.github.io/posts/oklab/>
4. **CIE.** (1976). CIELAB Color Space. *CIE 15:2004*.
5. **Adobe.** Leonardo: Adaptive Color Palette Generator. <https://leonardocolor.io/>
6. **contrast.tools.** <https://contrast.tools/>
7. **Brewer, C. A.** ColorBrewer 2.0. <https://colorbrewer2.org/>
8. **W3C.** CSS Color Module Level 4. <https://www.w3.org/TR/css-color-4/>
9. **ServiceNow.** Horizon Design System: Color Contrast. <https://horizon.servicenow.com/guidelines/accessibility/color-contrast>
10. **W3C Design Tokens Community Group.** Design Tokens Format Module. <https://tr.designtokens.org/>
11. **Stripe.** Designing accessible color systems. <https://stripe.com/blog/accessible-color-systems>
12. **Radix UI.** Radix Colors. <https://www.radix-ui.com/colors>

设计模式

1. **Gamma, E., Helm, R., Johnson, R., & Vlissides, J.** (1994). Design Patterns: Elements of Reusable Object-Oriented Software. *Addison-Wesley*.
2. **Buschmann, F., Meunier, R., Rohnert, H., Sommerlad, P., & Stal, M.** (1996). Pattern-Oriented Software Architecture, Volume 1: A System of Patterns. *Wiley*.
3. **Schmidt, D. C., Stal, M., Rohnert, H., & Buschmann, F.** (2000). Pattern-Oriented Software Architecture, Volume 2: Patterns for Concurrent and Networked Objects. *Wiley*.
4. **Fowler, M.** (2002). Patterns of Enterprise Application Architecture. *Addison-Wesley*.
5. **Evans, E.** (2003). Domain-Driven Design: Tackling Complexity in the Heart of Software. *Addison-Wesley*.
6. **Vernon, V.** (2013). Implementing Domain-Driven Design. *Addison-Wesley*.
7. **Martin, R. C.** (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. *Prentice Hall*.
8. **Nystrom, R.** (2014). Game Programming Patterns. *Genever Benning*.
9. **Fowler, M.** (2004). "Inversion of Control Containers and the Dependency Injection pattern." *martinfowler.com*. <https://martinfowler.com/articles/injection.html>

程序架构

1. **Richards, M., & Ford, N. (2020).** *Fundamentals of Software Architecture.* *O'Reilly Media.*
2. **Martin, R. C. (2017).** *Clean Architecture: A Craftsman's Guide to Software Structure and Design.* *Prentice Hall.*
3. **Evans, E. (2003).** *Domain-Driven Design: Tackling Complexity in the Heart of Software.* *Addison-Wesley.*
4. **Fowler, M. (2004).** "Inversion of Control Containers and the Dependency Injection pattern." *martinfowler.com.* <https://martinfowler.com/articles/injection.html>
5. **Electron Documentation.** Process Model. <https://www.electronjs.org/docs/latest/tutorial/process-model>

交互系统与状态管理

1. **Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994).** *Design Patterns: Elements of Reusable Object-Oriented Software.* *Addison-Wesley.*
2. **Harel, D. (1987).** "Statecharts: A Visual Formalism for Complex Systems." *Science of Computer Programming*, 8(3), 231–274.
3. **Apple.** Target-Action. *Developer Documentation Archive.* <https://developer.apple.com/library/archive/documentation/General/Conceptual/CocoaEnc>
4. **Qt Documentation.** QAction Class. <https://doc.qt.io/qt-6/qaction.html>
5. **GNU Emacs Lisp Reference Manual.** Keymaps. https://www.gnu.org/software/emacs/manual/html_node/elisp/Keymaps.html
6. **Visual Studio Code Documentation.** Commands. <https://code.visualstudio.com/api/extension-guides/command>
7. **Visual Studio Code Documentation.** User and Workspace Settings. <https://code.visualstudio.com/docs/getstarted/settings>
8. **Nystrom, R. (2014).** *Game Programming Patterns.* *Genever Benning.*
9. **Kleppmann, M. (2017).** *Designing Data-Intensive Applications.* *O'Reilly Media.*
10. **Redux Documentation.** Redux Style Guide. <https://redux.js.org/style-guide/>
11. **statecharts.dev.** Introduction to Statecharts. <https://statecharts.dev/>
12. **Vernon, V. (2013).** *Implementing Domain-Driven Design.* *Addison-Wesley.*

并发控制与资源管理

1. **Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994).** Design Patterns: Elements of Reusable Object-Oriented Software. *Addison-Wesley*.
2. **Electron Documentation.** Process Model.
<https://www.electronjs.org/docs/latest/tutorial/process-model>
3. **Electron Documentation.** Security.
<https://www.electronjs.org/docs/latest/tutorial/security>
4. **Electron Documentation.** `app.requestSingleInstanceLock()`.
<https://www.electronjs.org/docs/latest/api/app#apprequestsingleinstancelockadditional>
5. **Chromium Documentation.** Multi-process Architecture.
<https://www.chromium.org/developers/design-documents/multi-process-architecture/>

算法

1. **Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009).** Introduction to Algorithms (3rd ed.). *MIT Press*.
2. **Skiena, S. S. (2008).** The Algorithm Design Manual (2nd ed.). *Springer*.
3. **Knuth, D. E. (1997).** The Art of Computer Programming (Vols. 1–3). *Addison-Wesley*.
4. **Kleinberg, J., & Tardos, É. (2005).** Algorithm Design. *Addison-Wesley*.
5. **Tarjan, R. E. (1976).** “Edge-disjoint spanning trees and depth-first search.” *Acta Informatica*, 6, 171–185.

版本控制与发布工程

1. **Preston-Werner, T. (2013).** Semantic Versioning 2.0.0. <https://semver.org/>
2. **Humble, J., & Farley, D. (2010).** Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. *Addison-Wesley*.
3. **Lacan, O. (2017).** Keep a Changelog. <https://keepachangelog.com/>
4. **ISO/IEC/IEEE 26515:2018.** Systems and software engineering — Developing information for users in an agile environment.

拉尼亚利亚